ACSAC: Adaptive Chunk Size Actor-Critic with Causal Transformer Q-Network

Qian Chen[1] **Junqiao Zhao**[1] * **Hongtu Zhou**[1] **Hang Yu**[1]
Yanping Zhao[1] **Chen Ye**[1] **Guang Chen**[1]
[1]Tongji University

Abstract

Long-horizon, sparse-reward tasks pose a fundamental challenge for reinforcement learning, since single-step TD learning suffers from bootstrapping error accumulation across successive Bellman updates. Actor-critic methods with action chunking address this by operating over temporally extended actions, which reduce the effective horizon, enable fast value backups, and support temporally consistent exploration. However, existing methods rely on a fixed chunk size and therefore cannot adaptively balance reactivity against temporal consistency. A large fixed chunk size reduces responsiveness to new observations, while a small one produces incoherent motions, forcing task-specific tuning of the chunk size. To address this limitation, we propose **A**daptive **C**hunk **S**ize **A**ctor-**C**ritic (**ACSAC**). ACSAC leverages a causal Transformer critic to evaluate expected returns for action chunks of different sizes. At each chunk boundary, it adaptively selects the chunk size that maximizes the expected return, supporting flexible, state-dependent chunk sizes without task-specific tuning. We prove that the ACSAC Bellman operator is a contraction whose unique fixed point is the action-value function of the adaptive policy. Experiments on OGBench demonstrate that ACSAC achieves state-of-the-art performance on long-horizon, sparse-reward manipulation tasks across both offline RL and offline-to-online RL settings.

1 Introduction

Offline reinforcement learning (RL) trains policies from previously collected datasets without environment interaction [19], and the offline-to-online setting further refines the offline-pretrained policy through additional online interaction [29]. However, on long-horizon, sparse-reward tasks, single-step temporal difference (TD) learning suffers from bootstrapping error accumulation, since each update regresses toward the Q-network's own next-state estimate and small errors compound across many recursive Bellman updates [33]. A common remedy is to use multi-step returns, which accelerate value propagation by shifting the regression target further into the future, but they introduce an off-policy bias because the intermediate rewards are collected under the behavior policy rather than the current policy [20, 21]. Action chunking has emerged as a recent and effective response to these limitations.

Action chunking trains the critic and the policy on action sequences rather than single actions, which enables multi-step value backups without incurring off-policy bias, since the critic conditions on the full sequence [21]. Beyond these critic-side benefits, predicting full action sequences also produces temporally coherent behaviors that capture non-Markovian patterns in the data and improve exploration in sparse-reward settings [45, 21]. However, existing action-chunking actor-critic methods rely on a fixed chunk size, manually tuned per task and shared across all states [21, 22, 31, 16]. Yet the optimal balance between reactivity and temporal consistency is itself state-dependent [22, 23].

*Corresponding author

Preprint.

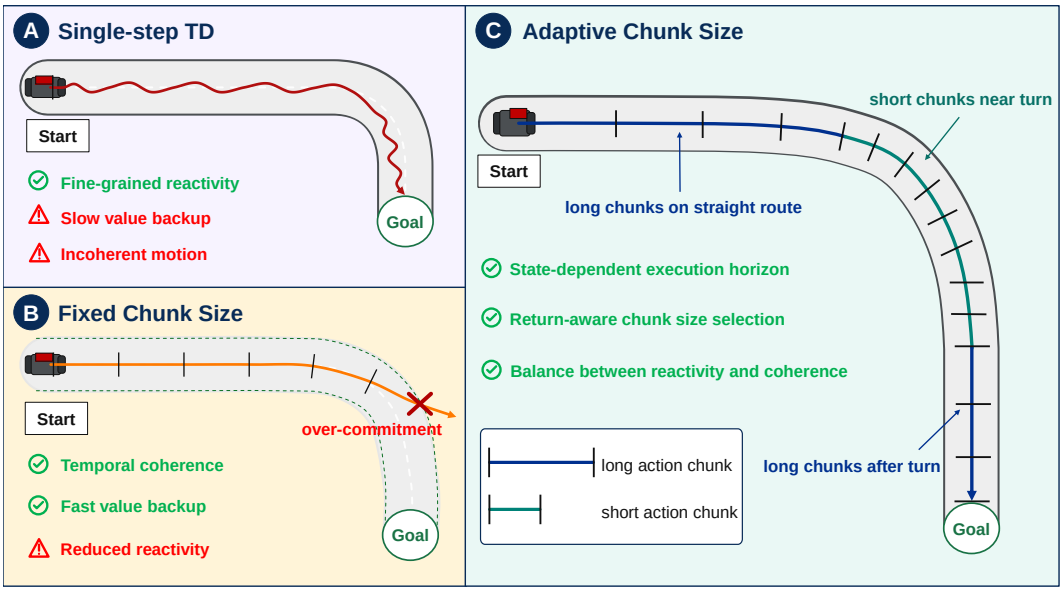

Figure 1: **Motivation for adaptive action chunk size.** **(A)** Single-step execution preserves fine-grained reactivity by replanning at every step, but suffers from slow value backups and produces incoherent motions. **(B)** A fixed chunk size improves motion coherence and accelerates value propagation, but its open-loop execution reduces reactivity within the chunk. In sensitive states such as turns, this over-commitment to a long chunk can drive the agent off the goal-reaching path, lowering the return. **(C)** ACSAC adaptively selects the execution horizon per state via return-aware chunk size selection, executing longer chunks on the straight route and shorter chunks near the turn, achieving a balance between reactivity and coherence.

Stable phases admit long open-loop chunks, whereas sensitive states demand frequent replanning. Over-committing to a long chunk in such states can drive the agent off the goal-reaching path (Figure 1) [22, 23].

To address this limitation, we propose **A**daptive **C**hunk **S**ize **A**ctor-**C**ritic (**ACSAC**), which adaptively selects the chunk size to maximize the critic's value estimate. At each replanning state, ACSAC samples multiple candidate action chunks of length H from an expressive flow BC policy. A **cross-horizon calibrated** causal Transformer critic evaluates the Q-value of every *prefix* (i.e., the first h actions of a chunk, $h = 1, \dots, H$) of each candidate. ACSAC then applies rejection sampling jointly over the candidate index and the prefix length, executes the highest-value prefix, and replans at the next chunk boundary. ACSAC thus retains the multi-step value backups and coherent exploration of action chunking, while the chunk size becomes state-dependent rather than fixed, adaptively balancing reactivity and temporal consistency at each state.

Our contributions can be summarized as follows: 1) We propose **ACSAC**, an action-chunking actor-critic that adaptively selects the chunk size per state, using a causal Transformer critic and joint rejection sampling over candidate index and prefix length. 2) We prove that ACSAC's prefix-conditioned Q-values are cross-horizon comparable, and that its Bellman operator is a contraction whose unique fixed point is the action-value function of the adaptive policy. 3) On long-horizon, sparse-reward manipulation tasks from OGBench [32], ACSAC outperforms single-step, multi-step, and fixed chunk size baselines in both offline and offline-to-online settings.

2 Related Work

Offline RL and offline-to-online RL. Offline RL aims to learn a policy from a fixed dataset without environment interaction [19]. The main challenge is the distributional shift between the behavior policy and the learned policy, which can cause value overestimation and suboptimal performance [9, 38]. Recently, expressive generative policies based on diffusion and flow matching [13, 24] have been widely adopted for their expressivity over Gaussian policies. Common policy extraction strategies for these generative policies include reparameterized gradients [40, 7, 34], weighted regression [27, 15, 6, 44], and rejection sampling [4, 11, 12]. In the offline-to-online setting, the offline-pretrained policy is further fine-tuned with online interactions [29], with techniques such as balanced sampling [18],

high update-to-data ratios [2], value calibration [30], and more [37, 42, 26, 46]. Our method uses the same algorithm for both offline and online training, simply adding online transitions to the offline dataset and applying none of the above specialized techniques.

Action chunking. Action chunking originated in imitation learning, where a policy predicts and executes a sequence of actions in an open-loop manner, improving robustness and capturing non-Markovian behavior [45, 5]. Recent RL methods have brought action chunking into actor-critic frameworks, where the critic evaluates whole action chunks and enables multi-step backup without off-policy bias. In the online setting with expert demonstrations, CQN-AS [35] learns a multi-level factorized critic on action chunks, and AC3 [41] builds on a DDPG-style framework to predict continuous action chunks. In the offline or offline-to-online setting, Q-chunking [21] runs RL at an action chunk level with a flow BC policy and rejection sampling. DQC [22] decouples the policy chunk size from the critic chunk size, with the policy predicting a shorter action chunk while retaining the value learning benefits of the chunked critic. MAC [31] combines an action-chunk dynamics model with rejection sampling from an expressive flow BC policy. DEAS [16] leverages action sequences for training critics with detached value learning and classification loss. CGQ [36] regularizes a single-step critic toward a chunked critic. All of the above RL methods rely on a fixed chunk size across states and tasks. However, the optimal chunk size may vary by state, and any fixed choice forces a single trade-off between reactivity and temporal consistency. Our method brings adaptive, state-dependent chunk size selection to RL, driven by the critic's value estimate.

Transformer-based Q-networks. Transformers have become strong backbones for the Q-network in RL. Q-Transformer [3] converts Q-function estimation into a discrete token sequence modeling problem, with per-dimension discretization treating each action dimension as a separate time step for autoregressive Q-value prediction. TQL [8] scales Transformer Q-learning through per-layer control of attention entropy that prevents attention collapse. While Q-Transformer and TQL apply Transformers to single-action Q-networks, recent work instead trains Transformer critics on action chunks for direct multi-step value evaluation. TOP-ERL [20] introduces a causal Transformer critic in episodic RL, with the policy outputting full trajectories via movement primitives. T-SAC [39] adapts a similar causal Transformer critic to step-based RL, conditioning the critic on short trajectory segments while keeping the actor single-step. SEAR [28] combines a causal Transformer critic with multi-horizon targets and random replanning during data collection. CO-RFT [14] applies a causal Transformer critic to fine-tune vision-language-action models with offline RL. None of these methods adapt the chunk size to state. Our method also adopts a causal Transformer critic, but uniquely exploits its prefix-conditioned outputs at all prefix lengths to support state-dependent chunk size selection during both training and inference.

3 Preliminaries

Problem formulation. We consider an infinite-horizon Markov decision process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, r, \rho, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A} \subseteq \mathbb{R}^d$ is the continuous action space of dimension d, $T(s'|s,a) : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ is the transition dynamics distribution, $r(s,a) : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function, $\rho \in \Delta(\mathcal{S})$ is the initial state distribution, and $\gamma \in [0,1)$ is the discount factor. Here $\Delta(\mathcal{X})$ denotes the set of probability distributions over a space $\mathcal{X}$. We assume access to a prior offline dataset $\mathcal{D}$ consisting of transition rollouts $\{(s,a,s',r)\}$ collected from $\mathcal{M}$. The goal is to find a policy $\pi(a|s) : \mathcal{S} \to \Delta(\mathcal{A})$ that maximizes the expected discounted return $J(\pi) := \mathbb{E}_{s_{t+1}\sim T(s_t,a_t), a_t\sim\pi(\cdot|s_t)}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t,a_t)\right]$. In the *offline* setting, the policy is learned entirely from the fixed dataset $\mathcal{D}$ without environment interactions; in the *offline-to-online* setting, the offline-pretrained policy is further fine-tuned with online interactions.

Chunk-based reinforcement learning. Standard TD-based methods learn a state-action value function $Q_\phi(s,a)$ by minimizing the single-step Bellman error:

$$\mathcal{L}^{\text{TD}}(\phi) = \mathbb{E}_{(s_t,a_t,r_t,s_{t+1})\sim\mathcal{D},\, a_{t+1}\sim\pi(\cdot|s_{t+1})}\left[\left(Q_\phi(s_t,a_t) - r_t - \gamma Q_{\bar{\phi}}(s_{t+1},a_{t+1})\right)^2\right], \quad (1)$$

where $Q_{\bar{\phi}}$ is a target network with delayed parameters $\bar{\phi}$. Each single-step backup propagates value only one step backward, slowing learning in long-horizon tasks. A common strategy is the multi-step return, which replaces the single-step target with $\sum_{\tau=0}^{n-1} \gamma^\tau r_{t+\tau} + \gamma^n Q_{\bar{\phi}}(s_{t+n}, a_{t+n})$ for some horizon $n \geq 1$ and allows for an n-fold speed-up in value propagation. However, the multi-step return introduces off-policy bias, since the discounted reward sum from the replay buffer may not

reflect the expected rewards under the current policy when the intermediate actions $a_{t+1}, \ldots, a_{t+n-1}$ are chosen by a different policy [20, 21].

Action chunking attains the value-propagation speedup of multi-step returns without their off-policy bias by extending RL to action sequences. An action chunk of length H starting at time t is $a_{t:t+H} := (a_t, a_{t+1}, \ldots, a_{t+H-1}) \in \mathcal{A}^H$, and the corresponding H-step discounted reward is $r_t^H := \sum_{\tau=0}^{H-1} \gamma^\tau r_{t+\tau}$. The chunked critic $Q_\phi(s_t, a_{t:t+H})$ is trained with:

$$\mathcal{L}^{\text{chunk}}(\phi) = \mathbb{E}_{(s_t, a_{t:t+H}, r_t^H, s_{t+H}) \sim \mathcal{D}} \left[\left(Q_\phi(s_t, a_{t:t+H}) - r_t^H - \gamma^H Q_{\bar{\phi}}(s_{t+H}, a_{t+H:t+2H}) \right)^2 \right], \quad (2)$$

where $a_{t+H:t+2H} \sim \pi(\cdot|s_{t+H})$. Crucially, unlike the multi-step return where the single-action critic $Q(s_t, a_t)$ is backed up with rewards generated by potentially off-policy actions, the chunked critic $Q(s_t, a_{t:t+H})$ conditions on the exact action sequence used to obtain the multi-step rewards r_t^H, eliminating the off-policy bias [21].

Flow policy. Flow matching [24, 25, 1] is a generative modeling technique that trains a velocity field to transform a noise distribution into a target data distribution. Given a target distribution $p(x) \in \Delta(\mathbb{R}^d)$, flow matching fits a time-dependent velocity field $v_\theta(u, x) : [0, 1] \times \mathbb{R}^d \to \mathbb{R}^d$ whose corresponding flow $\psi_\theta(u, x)$, defined by the ODE $\frac{d}{du}\psi_\theta(u, x) = v_\theta(u, \psi_\theta(u, x))$, transforms $\mathcal{N}(0, I_d)$ at $u = 0$ into $p(x)$ at $u = 1$, by minimizing:

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0 \sim \mathcal{N}(0, I_d),\, x_1 \sim p(x),\, u \sim \text{Unif}([0,1]),\, x_u = (1-u)x_0 + ux_1} \left[\| v_\theta(u, x_u) - (x_1 - x_0) \|_2^2 \right], \quad (3)$$

where $x_u := (1 - u)x_0 + ux_1$ is the linear interpolation between x_0 and x_1. To use flow matching for policy learning, we train a state-conditioned velocity field $v_\theta(u, s, a_z) : [0, 1] \times \mathcal{S} \times \mathbb{R}^d \to \mathbb{R}^d$ with the behavior cloning objective:

$$\mathcal{L}(\theta) = \mathbb{E}_{s,a \sim \mathcal{D},\, z \sim \mathcal{N}(0, I_d),\, u \sim \text{Unif}([0,1]),\, a_z = (1-u)z + ua} \left[\| v_\theta(u, s, a_z) - (a - z) \|_2^2 \right]. \quad (4)$$

We denote the ODE solution at $u = 1$ starting from noise z as $\pi_\theta(s, z) := \psi_\theta(1, s, z)$, which maps a noise vector $z \sim \mathcal{N}(0, I_d)$ to an action $a = \pi_\theta(s, z)$. Note that $\pi_\theta(s, z)$ is a deterministic function of z, but the stochasticity of z induces a conditional behavior distribution $\pi_\theta(\cdot|s)$. Compared to Gaussian policies, flow policies can model complex, multi-modal action distributions, making them particularly suited for offline RL where datasets often contain diverse behavior patterns [34, 31].

4 Method

ACSAC is an action-chunking actor-critic that adaptively selects the chunk size at each replanning state, rather than fixing one across all states as in prior methods. Throughout this section, H is the **maximum chunk size**, and for $h \in [H] := \{1, \ldots, H\}$ the length-h **prefix** of a chunk $a_{t:t+H} = (a_t, \ldots, a_{t+H-1})$ is its first h actions $a_{t:t+h} = (a_t, \ldots, a_{t+h-1})$. At each replanning state s_t, ACSAC samples N length-H candidate chunks $\{a_{t:t+H}^{(n)}\}_{n \in [N]}$ from a flow BC policy π_θ, evaluates all NH prefix-conditioned values $Q_\phi(s_t, a_{t:t+h}^{(n)})$ with a causal Transformer critic, and executes the prefix $a_{t:t+h^\star}^{(n^\star)}$ that achieves the joint argmax (Figure 2); the same extraction rule provides the bootstrap action prefix in the multi-step TD loss used to train Q_ϕ. We describe the critic architecture in Section 4.1, the multi-step TD objective in Section 4.2, and the flow BC policy with joint argmax extraction in Section 4.3.

4.1 Causal Transformer Critic Architecture

ACSAC's joint argmax requires the critic to evaluate action chunks of different lengths $a_{t:t+h}$ ($h \in [H]$) from a single state s_t and to produce values that are comparable across h. An MLP critic on action chunks consumes fixed-length inputs, so reusing it on shorter sub-prefixes incurs causal leakage [39]. A causal Transformer instead ingests $(s_t, a_{t:t+H})$ as a token sequence and outputs the H prefix-conditioned values $\{Q_\phi(s_t, a_{t:t+h})\}_{h \in [H]}$ jointly, with a causal attention mask that restricts position i to attend only to positions $j \leq i$.

This design has three consequences. First, $Q_\phi(s_t, a_{t:t+h})$ depends only on the prefix $a_{t:t+h}$ and not on future actions $a_{t+h:t+H}$, so each value is a valid estimate for executing exactly h actions and the MLP-style causal leakage is eliminated [39]. Second, the shared backbone produces sequence-aware

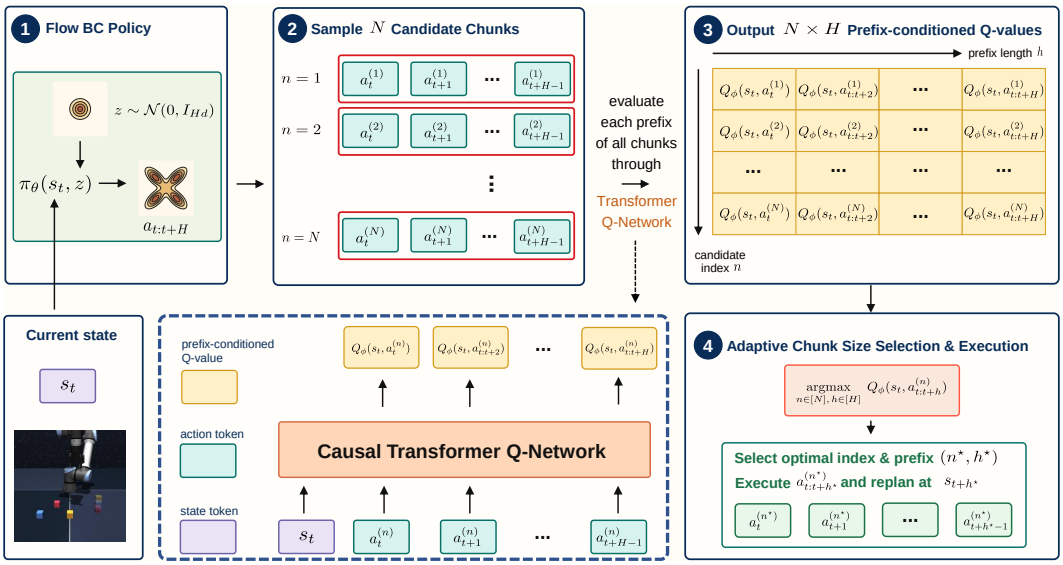

Figure 2: **Adaptive policy extraction in ACSAC.** At replanning state s_t, ACSAC samples N length-H chunks $\{a^{(n)}_{t:t+H}\}_{n\in[N]}$ from the flow BC policy $\pi_\theta(s_t, z)$ with $z^{(n)} \sim \mathcal{N}(0, I_{Hd})$, evaluates all prefix-conditioned values $Q_\phi(s_t, a^{(n)}_{t:t+h})$ for $(n, h) \in [N] \times [H]$, and executes $a^{(n^\star)}_{t:t+h^\star}$ where $(n^\star, h^\star) = \arg\max_{n\in[N],\, h\in[H]} Q_\phi(s_t, a^{(n)}_{t:t+h})$. The same extraction rule is used for bootstrap action sampling and deployment.

value estimates that capture the temporal structure within the chunk and enable fine-grained credit assignment across positions [20, 39]. Third, because all H values share a backbone and are jointly trained against per-horizon targets (Section 4.2), they live on a common return scale, making the joint argmax in $\pi_\star$ meaningful across different prefix lengths. We give a formal argument for prefix consistency and cross-horizon comparability in Appendix G.

4.2 Multi-Step TD Objective

We train the critic with a multi-step TD loss at every prefix length. For each $h \in [H]$, define the h-step return target

$$G_h(s_t, a_{t:t+h}) := \sum_{\tau=0}^{h-1} \gamma^\tau r_{t+\tau} + \gamma^h\, Q_{\bar{\phi}}\big(s_{t+h},\, \pi_\star(s_{t+h})\big), \tag{5}$$

which sums h on-policy rewards from the dataset and a bootstrap value from the target critic $Q_{\bar{\phi}}$ at the next state s_{t+h}. The critic loss averages the squared error against G_h across $h \in [H]$ in expectation over chunked transitions from the offline dataset or replay buffer:

$$\mathcal{L}(\phi) = \mathbb{E}_{(s_{t:t+H+1},\, a_{t:t+H},\, r_{t:t+H})\sim\mathcal{D}}\left[\frac{1}{H}\sum_{h=1}^{H}\big(Q_\phi(s_t, a_{t:t+h}) - G_h(s_t, a_{t:t+h})\big)^2\right]. \tag{6}$$

Each G_h propagates the bootstrap value at s_{t+h} back h steps to $Q_\phi(s_t, a_{t:t+h})$ in a single critic update [21]. Averaging gradients across prefix lengths further reduces gradient variance while preserving sparse reward signals, as formalized in Appendix G.5 [39, 20].

For the bootstrap value $Q_{\bar{\phi}}(s_{t+h}, \pi_\star(s_{t+h}))$, ACSAC samples N candidate chunks from π_θ at the next state s_{t+h}. The selected prefix $\pi_\star(s_{t+h})$ maximizes $Q_{\bar{\phi}}(s_{t+h}, a^{(n)}_{t+h:t+h+h'})$ over $(n, h') \in [N] \times [H]$. This generalizes EMaQ's expected-max Q operator [10], replacing the $\max$ over N candidate actions with a joint $\max$ over NH candidate prefixes. Appendix G shows that the resulting Bellman backup is a γ-contraction whose unique fixed point is the action-value function of $\pi_\star$.

4.3 Adaptive Policy Extraction

We use a flow BC policy $\pi_\theta(s, z)$ to generate candidate action chunks of length H from the offline dataset $\mathcal{D}$. π_θ is parameterized by a state-conditioned velocity field $v_\theta(u, s_t, a_z) : [0, 1] \times \mathcal{S} \times \mathbb{R}^{Hd} \to$

$\mathbb{R}^{Hd}$ trained with the flow-matching loss

$$\mathcal{L}(\theta) = \mathbb{E}_{\substack{z\sim\mathcal{N}(0,I_{Hd}),\,(s_t,a_{t:t+H})\sim\mathcal{D},\\ u\sim\text{Unif}([0,1]),\,a_z=(1-u)z+ua_{t:t+H}}} \left[\|v_\theta(u,s_t,a_z)-(a_{t:t+H}-z)\|_2^2\right]. \tag{7}$$

Fitting full-length action chunks lets π_θ capture non-Markovian behavior patterns in the offline data, supporting temporally coherent exploration in long-horizon tasks [21].

Given π_θ and the trained critic Q_ϕ, we extract ACSAC's policy $\pi_\star$ via rejection sampling, which implicitly enforces a behavior constraint with a closed-form bound on the KL divergence from π_θ [21] and is generally robust to hyperparameters, unlike alternatives that require tuning a behavior regularization coefficient [31]. Prior action-chunking methods [21, 22, 31] fix the chunk size h and apply rejection sampling: with $n^\star = \arg\max_{n\in[N]} Q_\phi(s_t, a^{(n)}_{t:t+h})$, the policy outputs $\pi(s_t) = a^{(n^\star)}_{t:t+h}$. ACSAC extends this single-axis rejection sampling to a joint search over NH candidates indexed by (n,h):

$$(n^\star,h^\star) = \arg\max_{n\in[N],\,h\in[H]} Q_\phi\left(s_t, a^{(n)}_{t:t+h}\right), \qquad \pi_\star(s_t) = a^{(n^\star)}_{t:t+h^\star}, \tag{8}$$

where $\{a^{(n)}_{t:t+H}\}_{n\in[N]}$ are obtained by drawing N noise vectors $z^{(n)}\sim\mathcal{N}(0,I_{Hd})$ and mapping them through the flow BC policy: $a^{(n)}_{t:t+H} = \pi_\theta(s_t, z^{(n)})$. The Transformer critic provides Q-values for all NH pairs, and $(n^\star,h^\star)$ is selected by a single argmax.

Intuitively, the selected execution horizon $h^\star$ reflects the current phase of the policy rollout at state s_t. When the Q-values of longer prefixes decline relative to shorter ones, the agent should replan after fewer steps to maintain reactivity. Conversely, when Q-values stay high or increase with longer prefixes, the state admits a coherent long-horizon plan and the agent benefits from a longer prefix to maintain temporal consistency. The selected execution horizon $h^\star \in [H]$ thus varies per state, with shorter $h^\star$ at sensitive states and longer $h^\star$ at states admitting coherent long-horizon plans. Importantly, the joint argmax across prefix lengths relies on ACSAC's prefix-conditioned Q-values lying on a common return scale. We prove this cross-horizon comparability in Appendix G.3 and verify it empirically in Section 5.3.

We provide pseudocode for the full offline pretraining and online fine-tuning procedures in Algorithm 1. Further implementation details are in Appendix B.

Algorithm 1 Adaptive Chunk Size Actor-Critic (ACSAC)

Require: Dataset $\mathcal{D}$, maximum chunk size H, rejection sampling size N, flow BC policy π_θ, causal Transformer critic Q_ϕ

// Offline training loop
while not converged **do**
  Sample chunked batch $\{(s_{t:t+H+1}, a_{t:t+H}, r_{t:t+H})\}\sim\mathcal{D}$
  Update flow BC policy π_θ with the flow-matching loss in Equation 7.
  Update causal Transformer critic Q_ϕ with the multi-step TD loss in Equation 6.

// Adaptive policy extraction from flow BC policy π_θ with rejection sampling
function $\pi_\star(s_t)$
  $z^{(n)}\sim\mathcal{N}(0,I_{Hd}),\ n\in[N]$
  $a^{(n)}_{t:t+H} = \pi_\theta(s_t, z^{(n)}),\ n\in[N]$
  $(n^\star,h^\star) \leftarrow \arg\max_{n\in[N],\ h\in[H]} Q_\phi(s_t, a^{(n)}_{t:t+h})$
  return $a^{(n^\star)}_{t:t+h^\star}$

// Online fine-tuning with adaptive replanning
Initialize $\mathcal{D}$ with offline data.
for every environment step t **do**
  if the previous selected chunk has been fully executed **then**
    $a^\star_{t:t+h^\star} \leftarrow \pi_\star(s_t)$
  Act with $a^\star_t$ and receive s_{t+1}, r_t.
  $\mathcal{D}\leftarrow\mathcal{D}\cup\{(s_t, a^\star_t, s_{t+1}, r_t)\}$
  Update π_θ via the flow-matching loss in Equation 7 using $\mathcal{D}$.
  Update Q_ϕ via the multi-step TD loss in Equation 6 using $\mathcal{D}$.

5 Experiments

We evaluate ACSAC on long-horizon, sparse-reward robotic manipulation tasks from OGBench [32]. Our experiments are designed to answer three questions. **Q1.** Does ACSAC improve offline-to-online RL performance over prior single-step, multi-step, and fixed chunk size methods? **Q2.** Does ACSAC's adaptive chunk size selection follow task phases, and are its prefix-conditioned Q-values calibrated and cross-horizon comparable? **Q3.** How do the design choices of ACSAC, including the maximum chunk size, the rejection sampling size, and adaptive chunk size selection itself, affect performance?

5.1 Experimental Setup

Environments and datasets. We evaluate ACSAC on OGBench manipulation tasks, which contain challenging long-horizon, sparse-reward robotic manipulation problems. We consider five domains with varying difficulties: `scene-sparse`, `puzzle-3x3-sparse`, `cube-double`, `cube-triple`, and `cube-quadruple`. Each domain contains five single-task variants, giving 25 tasks in total. We follow the `QC` dataset protocol for its five offline-to-online domains, including the 100M-transition dataset for `cube-quadruple` [21]. These domains are well suited for evaluating adaptive action chunking because they require both long-range value propagation and reactive manipulation at precise task phases. Additional domain metadata is provided in Appendix E.1.

Baselines. We compare ACSAC against single-step, multi-step, and fixed chunk size methods. The single-step baselines are `IQL` [17], `ReBRAC` [38], `FQL` [34], and `BFN` [21]. The multi-step baselines are `FQL-n` [21] and `BFN-n` [21], which use multi-step returns without learning both a chunked critic and a chunked policy. Finally, we compare against fixed chunk size methods, including `QC` [21], `QC-FQL` [21], and `DEAS` [16]. Implementation details for all baselines are summarized in Appendix B.

		puzzle-3x3-sparse (5 tasks)	scene-sparse (5 tasks)	cube-double (5 tasks)	cube-triple (5 tasks)	cube-quadruple (5 tasks)	overall (25 tasks)
Single-step	IQL	[0,0] [19,20] 0 → 20	[0,0] [39,39] 0 → 39	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [12,12] 0 → 12
	ReBRAC	[47,65] [100,100] 55 → 100	[7,16] [99,99] 11 → 99	[2,3] [28,32] 3 → 30	[0,0] [0,0] 0 → 0	[0,2] [20,20] 1 → 20	[13,15] [49,50] 14 → 50
	FQL	[99,100] [100,100] **100 → 100**	[55,60] [93,98] 57 → 95	[23,34] [75,76] 28 → 76	[1,2] [16,20] 1 → 18	[0,0] [0,6] 0 → 3	[36,38] [57,60] 37 → 58
	BFN	[96,99] [100,100] 98 → **100**	[82,87] [99,100] 85 → 99	[63,73] [78,81] 68 → 79	[3,5] [21,28] 4 → 23	[1,2] [12,13] 1 → 12	[50,52] [62,63] 51 → 63
Multi-step	FQL-n	[95,100] [100,100] 98 → **100**	[17,21] [64,76] 18 → 70	[7,13] [76,77] 11 → 77	[0,0] [0,1] 0 → 1	[6,8] [36,37] 7 → 37	[25,28] [56,58] 27 → 57
	BFN-n	[55,61] [89,92] 58 → 90	[50,64] [95,99] 57 → 97	[10,13] [62,69] 11 → 65	[0,1] [0,1] 0 → 0	[0,0] [0,0] 0 → 0	[24,26] [48,52] 25 → 50
Fixed Chunk Size	QC-FQL	[48,76] [100,100] 63 → **100**	[81,87] [99,100] 84 → 99	[32,47] [100,100] 39 → **100**	[3,5] [46,61] 4 → 53	[1,2] [76,77] 1 → **77**	[35,41] [84,88] 38 → 86
	QC	[99,100] [100,100] **100 → 100**	[79,89] [98,100] 84 → 99	[64,71] [96,99] 67 → 98	[3,10] [61,66] 6 → 64	[2,7] [72,75] 4 → 74	[50,53] [86,87] 52 → 86
	DEAS	[92,98] [100,100] 95 → **100**	[91,95] [91,98] 93 → 95	[50,64] [83,86] 56 → 85	[4,6] [66,73] 4 → 69	[16,21] [20,25] **18** → 22	[50,56] [72,77] 53 → 74
Adaptive Chunk Size	ACSAC	[100,100] [100,100] **100 → 100**	[99,100] [100,100] **99 → 100**	[82,87] [99,100] **84 → 100**	[14,25] [95,99] **19 → 97**	[1,10] [71,74] 5 → 73	[59,64] [93,94] **61 → 94**

Table 1: **Summary table for OGBench offline-to-online RL results.** For each cell, we report the offline performance after 1M of training steps and then the online performance after 1M of additional online steps. The best method(s) for each column is highlighted in bold and color. ACSAC consistently outperforms all prior single-step, multi-step, and fixed chunk size action-chunking baselines at the end of both the offline training and the online training. See the full per-task results in Table 6 (complete table) and Figure 6 (individual training curves). All values are means over 4 seeds with 95% confidence intervals.

5.2 Main Results

We report the main offline-to-online RL results in Table 1. ACSAC achieves the best overall performance across the five-domain suite, outperforming single-step, multi-step, and fixed chunk size action-chunking baselines in both the offline phase and the offline-to-online phase. The improvement is most visible on the long-horizon `cube-triple` and `scene-sparse` domains, where adaptive chunk size selection is beneficial when an episode alternates between coarse transport and precise manipulation.

Fixed chunk size methods such as QC, QC-FQL, and DEAS substantially improve over single-step and multi-step flow baselines on the long-horizon cube domains, which alone demonstrates the importance of action chunking. A fixed chunk size, however, imposes a single trade-off between reactivity and temporal consistency throughout the entire episode. ACSAC adaptively selects the execution horizon at each replanning state, retaining the fast value backups of long chunks while avoiding unnecessary open-loop execution in states that require precise feedback.

The main exception is cube-quadruple, where fixed chunk size methods remain highly competitive online. A plausible reason is that the default maximum chunk size is still short relative to the temporal structure of a four-object manipulation task, so adaptivity alone cannot cover a full behavior segment. Complete per-task results and training curves are in Appendix F.2.

5.3 Qualitative and Quantitative Analyses

Distribution of chunk size decisions. We visualize in Figure 3 the mean executed chunk size at each observation timestep on a representative cube-double pick-and-place task, averaged over 50 episodes of the online checkpoint. The curve closely follows the semantic phases of the task: ACSAC commits to large chunks early ($t < 5$, mean ≈ 4) for the coarse approach toward the cube, intermediate chunks during grasp and transport ($5 \leq t \leq 25$, mean ≈ 3), and progressively smaller chunks toward precise placement, reaching mean ≈ 1 around $t = 33$ as the cube is aligned at the target. This adaptive pattern mirrors the qualitative finding of Liang et al. [23] for vision-language-action models: larger chunks enable fast, coarse movements while smaller chunks with high-frequency replanning ensure precise control during the critical grasping and placement phases. Unlike Liang et al. [23], where chunk size is driven by predicted action entropy, in ACSAC it emerges directly from the learned prefix-conditioned Q-values.

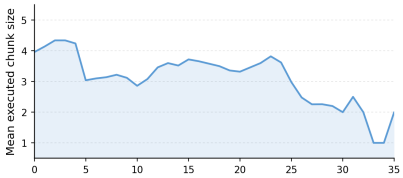

Figure 3: **Distribution of chunk size decisions from ACSAC.** Mean executed chunk size at each observation timestep on a representative cube-double pick-and-place task, averaged over 50 episodes of the online checkpoint.

Prefix-Q calibration and cross-horizon comparability. We test whether ACSAC's prefix-conditioned Q-values $\hat{Q}$ are calibrated against realized returns and comparable across horizons. We collect 50 rollouts of the deployed adaptive policy and of five fixed-h controls, each selecting only among N candidates of length h. Along all rollouts we record $\hat{Q}$ and discounted Monte-Carlo returns $\hat{G}$, then plot the mean $\hat{Q}$ in equal-frequency bins of $\hat{G}$. Deviation from the diagonal $\hat{Q} = \hat{G}$ quantifies over- or under-estimation. As shown in Figure 4, all six curves track the diagonal closely and largely coincide along the return range. This indicates that ACSAC's prefix Q-values are individually calibrated and mutually comparable across h. The joint $\arg\max$ over the $N \times H$ candidates is therefore a principled deployment rule.

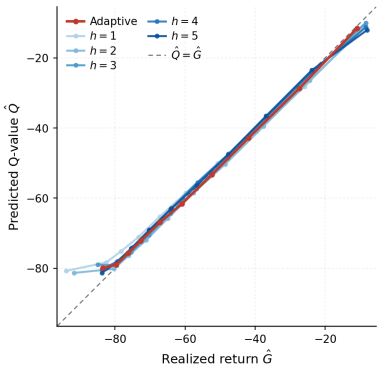

Figure 4: **Prefix-Q calibration.** Binned predicted Q-value $\hat{Q}$ versus realized Monte-Carlo return $\hat{G}$ over 50 rollouts of the online checkpoint, for the deployed adaptive policy and five fixed-h controls.

5.4 Ablation Study

We ablate the main design choices of ACSAC on the cube-double and cube-triple domains.

In Figure 5 (top), we sweep the maximum chunk size $H \in \{1, 3, 5, 7\}$ around the default $H{=}5$. $H{=}1$ fails on both domains, confirming that single-step execution cannot provide the value propagation needed for these sparse-reward tasks. $H{=}3$ and $H{=}5$ perform comparably on both domains, with $H{=}3$ slightly ahead on cube-triple. Increasing to $H{=}7$ hurts cube-triple, consistent with the observation that overly long chunks make the behavior policy and critic harder to learn [21, 22, 31]. We use $H{=}5$ in all other experiments to allow the adaptive policy a broader range of execution horizons.

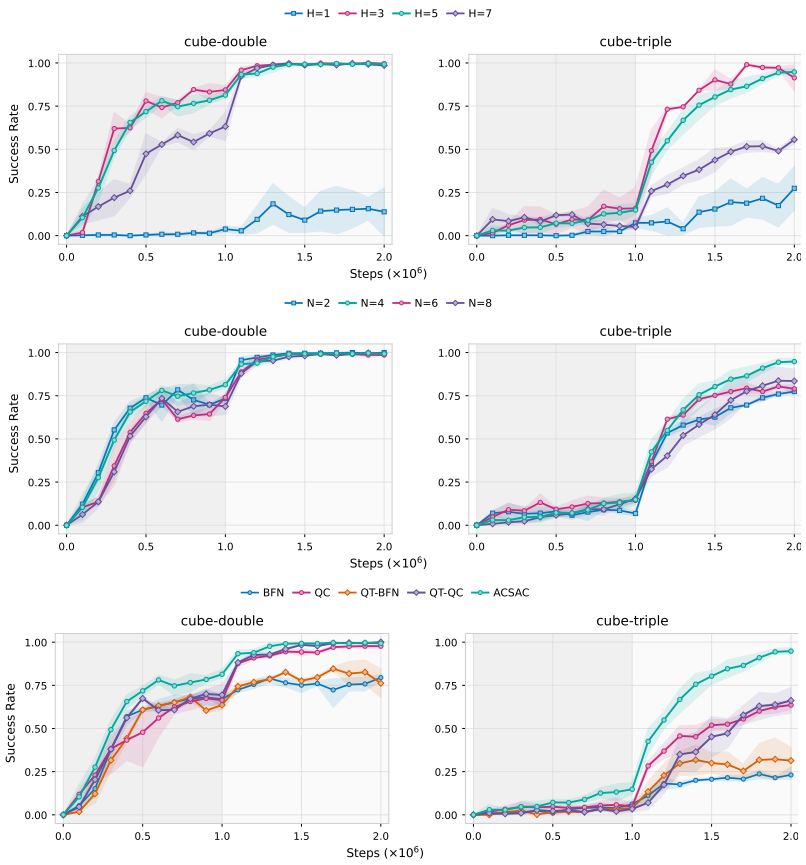

Figure 5: **Ablation studies.** *Top:* maximum chunk size H sweep. *Middle:* rejection sampling size N sweep. *Bottom:* same-architecture controls QT-BFN and QT-QC replacing the MLP critics in BFN and QC with ACSAC's causal Transformer critic. Curves aggregate five tasks per domain. The first 1M steps are offline and the next 1M steps are online.

In Figure 5 (middle), we vary the rejection sampling size $N \in \{2, 4, 6, 8\}$ around the default $N{=}4$. $N{=}2$ weakens value-based action selection, especially online on `cube-triple`, while increasing N to 6 or 8 saturates with no consistent gain. We use $N{=}4$ as the best overall trade-off.

Finally, in Figure 5 (bottom), we isolate the contribution of the causal Transformer critic with two same-architecture controls. QT-QC and QT-BFN replace the MLP critics of QC and BFN with ACSAC's causal Transformer critic, keeping every other component unchanged from Li et al. [21]. Both controls improve over the original baselines but do not close the gap to ACSAC, most clearly on `cube-triple`. This indicates that ACSAC's advantage is not merely a Transformer-critic effect. The multi-horizon TD objective in Equation 6 and the joint $\arg\max$ over (n, h) in Equation 8 are essential.

6 Conclusion

We propose ACSAC, which enables adaptive chunk selection through cross-horizon calibrated prefix value learning. By learning prefix-conditioned Q-values on a shared discounted-return scale, ACSAC allows executable prefixes with different horizons to be compared consistently, thereby supporting adaptive replanning behavior without manual chunk-size tuning. A causal Transformer critic trained with a multi-step TD objective produces these prefix-conditioned values. A flow BC policy paired with rejection sampling over the joint (n, h) axes yields the executed chunk. On long-horizon, sparse-reward OGBench manipulation tasks, ACSAC outperforms single-step, multi-step, and fixed chunk size baselines in both offline and offline-to-online settings. A promising future direction is to integrate state-dependent chunk sizes with vision-language-action models. Recent work shows that action-sequence critics scale to large VLAs in both simulation and real-world experiments [16, 14], bringing RL closer to real-world applications.

References

[1] Michael S. Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=li7qeBbCR1t.

[2] Philip J. Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning*, pages 1577–1594. PMLR, 2023.

[3] Yevgen Chebotar, Quan Ho Vuong, Karol Hausman, Fei Xia, Yao Lu, Alex Irpan, Aviral Kumar, Tianhe Yu, Alexander Herzog, Karl Pertsch, Keerthana Gopalakrishnan, Julian Ibarz, Sergey Levine, Adrian Salazar, and Chelsea Finn. Q-transformer: Scalable offline reinforcement learning via autoregressive Q-functions. In *Conference on Robot Learning*, pages 3909–3928. PMLR, 2023.

[4] Huayu Chen, Cheng Lu, Chengyang Ying, Hang Su, and Jun Zhu. Offline reinforcement learning via high-fidelity generative behavior modeling. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=42zs3qa2kpy.

[5] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025. doi: 10.1177/02783649241273668.

[6] Shutong Ding, Ke Hu, Zhenhao Zhang, Kan Ren, Weinan Zhang, Jingyi Yu, Jingya Wang, and Ye Shi. Diffusion-based reinforcement learning via q-weighted variational policy optimization. In *Advances in Neural Information Processing Systems*, volume 37, pages 53945–53968, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/6111371a868af8dcfba0f96ad9e25ae3-Abstract-Conference.html.

[7] Zihan Ding and Chi Jin. Consistency models as a rich and efficient policy class for reinforcement learning. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=v8jdwkUNXb.

[8] Perry Dong, Kuo-Han Hung, Alexander Swerdlow, Dorsa Sadigh, and Chelsea Finn. TQL: Scaling q-functions with transformers by preventing attention collapse. *arXiv preprint arXiv:2602.01439*, 2026.

[9] Scott Fujimoto and Shixiang Shane Gu. A minimalist approach to offline reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 34, pages 20132–20145, 2021.

[10] Seyed Kamyar Seyed Ghasemipour, Dale Schuurmans, and Shixiang Shane Gu. EMaQ: Expected-max q-learning operator for simple yet effective offline and online RL. In *International Conference on Machine Learning*, pages 3682–3691. PMLR, 2021.

[11] Philippe Hansen-Estruch, Ilya Kostrikov, Michael Janner, Jakub Grudzien Kuba, and Sergey Levine. IDQL: Implicit q-learning as an actor-critic method with diffusion policies. *arXiv preprint arXiv:2304.10573*, 2023.

[12] Longxiang He, Li Shen, Junbo Tan, and Xueqian Wang. AlignIQL: Policy alignment in implicit q-learning through constrained optimization. *arXiv preprint arXiv:2405.18187*, 2024.

[13] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

[14] Dongchi Huang, Zhirui Fang, Tianle Zhang, Yihang Li, Lin Zhao, and Chunhe Xia. CO-RFT: Efficient fine-tuning of vision-language-action models through chunked offline reinforcement learning. *arXiv preprint arXiv:2508.02219*, 2025.

[15] Bingyi Kang, Xiao Ma, Chao Du, Tianyu Pang, and Shuicheng Yan. Efficient diffusion policies for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 67195–67212, 2023.

[16] Changyeon Kim, Haeone Lee, Younggyo Seo, Kimin Lee, and Yuke Zhu. DEAS: Detached value learning with action sequence for scalable offline RL. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=bVTaAXeBmE.

[17] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=68n2s9ZJWF8.

[18] Seunghyun Lee, Younggyo Seo, Kimin Lee, Pieter Abbeel, and Jinwoo Shin. Offline-to-online reinforcement learning via balanced replay and pessimistic q-ensemble. In *Conference on Robot Learning*, pages 1702–1712. PMLR, 2022.

[19] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.

[20] Ge Li, Dong Tian, Hongyi Zhou, Xinkai Jiang, Rudolf Lioutikov, and Gerhard Neumann. TOP-ERL: Transformer-based off-policy episodic reinforcement learning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=N4NhVN3Oph.

[21] Qiyang Li, Zhiyuan Zhou, and Sergey Levine. Reinforcement learning with action chunking. In *Advances in Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=XUks1Y96NR.

[22] Qiyang Li, Seohong Park, and Sergey Levine. Decoupled q-chunking. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=aqGNdZQL9l.

[23] Yuanchang Liang, Xiaobo Wang, Kai Wang, Shuo Wang, Xiaojiang Peng, Haoyu Chen, David Kim Huat Chua, and Prahlad Vadakkepat. Adaptive action chunking at inference-time for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026.

[24] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=PqvMRDCJT9t.

[25] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=XVjTT1nw5z.

[26] Xu-Hui Liu, Tian-Shuo Liu, Shengyi Jiang, Ruifeng Chen, Zhilong Zhang, Xinwei Chen, and Yang Yu. Energy-guided diffusion sampling for offline-to-online reinforcement learning. In *International Conference on Machine Learning*, pages 31541–31565. PMLR, 2024. URL https://proceedings.mlr.press/v235/liu24ao.html.

[27] Cheng Lu, Huayu Chen, Jianfei Chen, Hang Su, Chongxuan Li, and Jun Zhu. Contrastive energy prediction for exact energy-guided diffusion sampling in offline reinforcement learning. In *International Conference on Machine Learning*, pages 22825–22855. PMLR, 2023.

[28] C. F. Maximilian Nagy, Onur Celik, Emiliyan Gospodinov, Florian Seligmann, Weiran Liao, Aryan Kaushik, and Gerhard Neumann. SEAR: Sample efficient action chunking reinforcement learning. *arXiv preprint arXiv:2603.01891*, 2026.

[29] Ashvin Nair, Abhishek Gupta, Murtaza Dalal, and Sergey Levine. AWAC: Accelerating online reinforcement learning with offline datasets. *arXiv preprint arXiv:2006.09359*, 2020.

[30] Mitsuhiko Nakamoto, Simon Zhai, Anikait Singh, Max Sobol Mark, Yi Ma, Chelsea Finn, Aviral Kumar, and Sergey Levine. Cal-QL: Calibrated offline RL pre-training for efficient online fine-tuning. In *Advances in Neural Information Processing Systems*, volume 36, pages 62244–62269, 2023.

[31] Kwanyoung Park, Seohong Park, Youngwoon Lee, and Sergey Levine. Scalable offline model-based RL with action chunks. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=WXGb9unEHo.

[32] Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking offline goal-conditioned RL. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=M992mjgKzI.

[33] Seohong Park, Kevin Frans, Deepinder Mann, Benjamin Eysenbach, Aviral Kumar, and Sergey Levine. Horizon reduction makes RL scalable. In *Advances in Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=hguaupzLCU.

[34] Seohong Park, Qiyang Li, and Sergey Levine. Flow q-learning. In *International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=KVf2SFL1pi.

[35] Younggyo Seo and Pieter Abbeel. Coarse-to-fine q-network with action sequence for data-efficient reinforcement learning. In *Advances in Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=VoFXUNc9Zh.

[36] Gwanwoo Song, Kwanyoung Park, and Youngwoon Lee. Chunk-guided q-learning. *arXiv preprint arXiv:2603.13971*, 2026.

[37] Yuda Song, Yifei Zhou, Ayush Sekhari, J. Andrew Bagnell, Akshay Krishnamurthy, and Wen Sun. Hybrid RL: Using both offline and online data can make RL efficient. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=yyBis80iUuU.

[38] Denis Tarasov, Vladislav Kurenkov, Alexander Nikulin, and Sergey Kolesnikov. Revisiting the minimalist approach to offline reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 11592–11620, 2023.

[39] Dong Tian, Onur Celik, and Gerhard Neumann. Chunking the critic: A transformer-based soft actor-critic with N-step returns. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=rb5eTktqbc.

[40] Zhendong Wang, Jonathan J. Hunt, and Mingyuan Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=AHvFDPi-FA.

[41] Jiarui Yang, Bin Zhu, Jingjing Chen, and Yu-Gang Jiang. Actor-critic for continuous action chunks: A reinforcement learning framework for long-horizon robotic manipulation with sparse reward. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(22):18692–18700, 2026. doi: 10.1609/aaai.v40i22.38937.

[42] Zishun Yu and Xinhua Zhang. Actor-critic alignment for offline-to-online reinforcement learning. In *International Conference on Machine Learning*, pages 40452–40474. PMLR, 2023.

[43] Yang Yue, Rui Lu, Bingyi Kang, Shiji Song, and Gao Huang. Understanding, predicting and better resolving Q-value divergence in offline-RL. In *Advances in Neural Information Processing Systems*, volume 36, pages 60247–60277, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/bd6bb13e78da078d8adcabbe6d9ca737-Abstract-Conference.html.

[44] Shiyuan Zhang, Weitong Zhang, and Quanquan Gu. Energy-weighted flow matching for offline reinforcement learning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=HA0oLUvuGI.

[45] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems*, 2023.

[46] Zhiyuan Zhou, Andy Peng, Qiyang Li, Sergey Levine, and Aviral Kumar. Efficient online reinforcement learning fine-tuning need not retain offline data. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=HN0CYZbAPw.

A Limitations

We highlight three limitations of ACSAC and corresponding directions for future work. First, larger H aggravates the multi-modality of behavior chunks [16] and larger N scales the per-step cost, so jointly scaling (H, N) together with the Transformer critic capacity is a natural next step. Second, our evaluation focuses on long-horizon manipulation from OGBench and excludes navigation domains such as `antmaze` and `humanoidmaze`, where action-chunked methods are known to be less effective due to highly reactive control and fine-grained trajectory stitching [36, 31, 22]. Our theoretical analysis also assumes deterministic transitions (Appendix G) [22], leaving stochastic environments as a promising extension. Third, we do not evaluate ACSAC on real-world robotic deployment or on large vision-language-action backbones, but recent work shows that chunked value learning scales to such settings [16, 14], presenting a natural extension.

B Implementation Details

Online fine-tuning. For offline-to-online RL, we simply add online transitions to the dataset, without distinguishing them from the offline transitions. We continue to train each method with the same objective as in offline training, following Park et al. [34] and Li et al. [21].

Flow matching. Following Li et al. [21], each action-chunking baseline trains a flow BC policy on the dataset, parameterized by a state-conditioned velocity field v_θ trained with the flow-matching loss in Equation 7. At inference, an action chunk is generated by Euler integration of v_θ over F flow steps from $a^0 = z \sim \mathcal{N}(0, I_{Hd})$. Single-action methods such as `FQL` and `BFN` train the velocity field on individual actions instead of action chunks.

Causal Transformer. ACSAC ingests $(s_t, a_t, a_{t+1}, \ldots, a_{t+H-1})$ as a token sequence and outputs H heads, $[Q_\phi(s_t, a_t), Q_\phi(s_t, a_{t:t+2}), \ldots, Q_\phi(s_t, a_{t:t+H})]$, in one forward pass. A causal attention mask ensures that the h-th head depends only on $a_{t:t+h}$. Pre-LayerNorm is applied before every attention and feed-forward sublayer.

Value learning. Following standard practice, all methods train two Q-networks for stability. ACSAC takes the minimum of the two Q-values [9] for both the bootstrap target and the policy extraction, while each baseline retains its original aggregation rule. ACSAC's bootstrap target uses the current online critic with the gradient stopped. SEEM [43] shows that LayerNorm bounds the critic's NTK, allowing stable training even when online and target networks share parameters.

C Baselines

FQL. `FQL` [34] is a behavior regularization-based offline / offline-to-online method that distills a one-step noise-conditioned policy from a flow BC policy with a single-step MLP critic.

FQL-n. `FQL-n` [21] is a multi-step return variant of `FQL` that replaces the single-step TD target with a multi-step return at the chunked horizon.

QC-FQL and QC. `QC-FQL` [21] extends `FQL` to the chunked action space. `QC` [21] replaces `QC-FQL`'s distilled one-step policy with rejection sampling over the flow BC policy. We reproduce both with the official implementation released by Li et al. [21].

BFN and BFN-n. `BFN` [21] runs `QC`'s rejection sampling extraction in the original single-action space. `BFN-n` [21] additionally uses a multi-step TD target.

DEAS. `DEAS` [16] extends action-chunked offline RL with detached value learning, distributional RL with fixed support, and dual discount factors for stable training. We reproduce `DEAS` with the official implementation released by Kim et al. [16], following its cube-task hyperparameters and Song et al. [36]'s per-task entries for `scene-sparse` and `puzzle-3x3-sparse`.

QT-QC and QT-BFN. Same-architecture controls that replace `QC` and `BFN`'s MLP critics with ACSAC's causal Transformer critic, keeping every other component unchanged from Li et al. [21].

D Hyperparameters

D.1 Shared hyperparameters

We report in Table 2 the hyperparameters shared across all methods in our experiments.

Hyperparameter	Value
Optimizer	Adam
Learning rate	3×10^{-4}
Batch size	256
Discount factor γ	0.99
Target network update rate τ (when used)	5×10^{-3}
UTD ratio	1
Number of flow steps F	10
Critic ensemble size K	2
Offline training steps	10^6
Online environment steps	10^6
Evaluation interval	10^5 steps
Evaluation episodes	50
MLP network width	512
MLP network depth	4 hidden layers

Table 2: **Shared hyperparameters across all methods.**

D.2 Task-specific hyperparameters

We report in Table 3 the per-task hyperparameter choices of each method.

Domain	ACSAC (H, N)	QC-FQL (α, h)	QC (h, N)	DEAS $(\alpha, h, \gamma_1, \gamma_2, s)$
`scene-sparse-*`	$(5, 4)$	$(300, 5)$	$(5, 32)$	$(3, 4, 0.99, 0.999, u)$
`puzzle-3x3-sparse-*`	$(5, 4)$	$(300, 5)$	$(5, 64)$	$(3, 8, 0.9, 0.99, u)$
`cube-double-*`	$(5, 4)$	$(300, 5)$	$(5, 32)$	$(300, 4, 0.9, 0.995, d)$
`cube-triple-*`	$(5, 4)$	$(100, 5)$	$(5, 32)$	$(1, 4, 0.9, 0.995, d)$
`cube-quadruple-100M-*`	$(5, 8)$	$(100, 5)$	$(5, 32)$	$(1, 4, 0.9, 0.995, d)$

Table 3: **Task-specific hyperparameters.** The layout follows Song et al. [36]'s all-method task-specific hyperparameter table. For ACSAC, H is the maximum chunk size and N is the rejection sampling size. The Transformer critic uses n_{layer}=2 attention layers, n_{head}=8 heads, and per-head dimension d_{head}=16 across all tasks. For `QC-FQL`, α is the behavior regularization coefficient and h is the chunk size. For `QC`, h is the chunk size and N is the rejection sampling size. For `DEAS`, d and u denote data-centric and universal support, respectively.

E Experiment Details

E.1 Environments, Tasks, and Datasets

OGBench [32]. We evaluate ACSAC on five long-horizon OGBench manipulation domains: `scene-sparse`, `puzzle-3x3-sparse`, `cube-double`, `cube-triple`, and `cube-quadruple`. Following Li et al. [21], `scene-sparse` and `puzzle-3x3-sparse` sparsify the rewards of OGBench's `scene-play` and `puzzle-3x3-play` to $\{-1, 0\}$, where -1 is given when the task is incomplete and 0 when it is completed. Each domain contains five single-task variants, giving 25 OGBench tasks

in total. These domains are particularly suitable for studying action chunking because successful behavior requires both long-range value propagation and temporally coherent action sequences. Dataset size, episode length, and action dimension for each domain are listed in Table 4.

`scene-sparse`**.** This domain contains a drawer, a window, a cube, and two button locks that control whether the drawer and the window can be opened. Tasks require multi-stage manipulation, such as unlocking the scene, moving the drawer or window, placing the cube, and relocking the scene. The long sequence of necessary sub-behaviors makes this domain sensitive to slow value propagation and incoherent short-horizon exploration.

`puzzle-3x3-sparse`**.** This domain contains a 3×3 grid of buttons. Pressing a button flips its own state and the states of adjacent buttons. The task is to reach a target color configuration. Because local actions can have coupled downstream effects, the domain tests whether a policy can execute coherent short plans while still replanning when the current prefix becomes unfavorable.

`cube-double/triple/quadruple`**.** These domains require a robot arm to move two, three, or four cubes to target locations. The reward depends on the number of cubes that remain incorrectly placed, and an episode terminates when all cubes are correctly placed. The `cube-triple` and `cube-quadruple` domains are especially challenging in the offline-to-online setting because solving them often requires efficient online exploration after offline pretraining.

Domain	Dataset Size	Episode Length	Action Dim.
`scene-sparse-*`	1M	750	5
`puzzle-3x3-sparse-*`	1M	500	5
`cube-double-*`	1M	500	5
`cube-triple-*`	3M	1000	5
`cube-quadruple-100M-*`	100M	1000	5

Table 4: **Domain metadata.** Dataset size is measured in transitions. All OGBench manipulation domains use a five-dimensional action space corresponding to end-effector translation, yaw, and gripper opening.

E.2 Evaluation Protocol

Offline-to-online evaluation. The main offline-to-online RL results in Table 1 and the per-task results in Table 6 use this protocol. Following Park et al. [34], Li et al. [21], agents are pretrained for 1M offline gradient steps and fine-tuned for 1M online environment steps, with success rates reported at 1M and 2M steps. All results are averaged over 4 random seeds, with 95% confidence intervals computed via 5000-sample stratified bootstrap resampling [21].

Offline evaluation. The offline RL results in Table 7 use this protocol. Following Park et al. [32, 34], we train each method for 1M offline gradient steps, evaluate every 100K steps over 50 episodes, and report the average success rate across the last three evaluation epochs (800K, 900K, 1M).

Results from prior works. For Table 1, the `IQL`, `ReBRAC`, `FQL`, `BFN`, `FQL-n`, `BFN-n`, `QC`, and `QC-FQL` entries are taken from `QC`'s released plot data[2]. For Table 7, the `FQL`, `FQL-n`, `QC-FQL`, `DEAS`, `DQC`, and `CGQ` entries are taken from Song et al. [36]. Otherwise, we implement the baselines in our codebase and evaluate them under our protocol.

F Additional Experimental Results

F.1 Computational Costs

We run all experiments on NVIDIA RTX 3090 GPUs. Table 5 reports parameter counts and per-step runtimes on `cube-triple-task1`.

[2] `https://github.com/ColinQiyangLi/qc/tree/main/plot_data`

Parameter count. DEAS is counted under its default cube-triple reproduction configuration [16]. ACSAC has the smallest parameter count because its shallow Transformer critic is shared across all prefix lengths rather than instantiating a separate critic per chunk size.

Per-step runtime. QC-FQL has comparable runtime to FQL and BFN for both offline and online training. QC is slower offline because it samples 32 actions per training example, while BFN samples 4. ACSAC shares QC's expected-max sampling backbone and adds only a shallow (n_{layer}=2, n_{embd}=128) Transformer, so its offline runtime is on par with QC. DEAS skips rejection sampling and is therefore substantially faster offline. For online training, QC and ACSAC are only 30–50% more expensive than FQL, BFN, and QC-FQL.

Method	Parameter Count	Offline Run-time (ms)	Online Run-time (ms)
FQL	4,911,630	2.84	10.26
BFN	4,094,473	3.19	11.82
QC	4,155,933	11.50	15.37
QC-FQL	4,993,590	4.06	11.55
DEAS	5,231,137	3.00	5.42
ACSAC	2,455,065	12.50	13.76

Table 5: **Parameter count and per-step runtime on cube-triple-task1.** Offline measures one agent training step. Online measures one agent training step plus one environment step. Parameter counts are measured on cube-triple-task1 (46-dim observation, 5-dim action).

F.2 Full Offline-to-Online RL Results for OGBench

Table 6 reports per-task offline-to-online RL results aligned with the domain-level summary in Table 1. The ACSAC and DEAS columns are reproduced under our unified protocol. Figure 6 provides summary plots by domain (matching Table 1) followed by per-task curves (matching Table 6).

		IQL	ReBRAC	FQL	BFN	FQL-n	BFN-n	QC-FQL	QC	DEAS	ACSAC
puzzle-3x3-sparse	task1	[0,0] [97,100] 0 → 99	[90,100] [100,100] 96 → **100**	[98,100] [100,100] 99 → **100**	[97,100] [100,100] 98 → **100**	[98,100] [100,100] **100** → **100**	[86,91] [100,100] 88 → **100**	[100,100] [100,100] **100** → **100**	[100,100] [100,100] **100** → **100**	[100,100] [100,100] **100** → **100**	[100,100] [100,100] **100** → **100**
	task2	[0,0] [0,0] 0 → 0	[46,86] [100,100] 70 → **100**	[98,100] [100,100] 99 → **100**	[96,100] [100,100] 98 → **100**	[100,100] [100,100] **100** → **100**	[66,77] [100,100] 72 → **100**	[60,100] [100,100] 86 → **100**	[100,100] [100,100] **100** → **100**	[95,100] [100,100] 98 → **100**	[100,100] [100,100] **100** → **100**
	task3	[0,0] [0,0] 0 → 0	[6,56] [100,100] 27 → **100**	[98,100] [100,100] **100** → **100**	[89,100] [100,100] 95 → **100**	[98,100] [100,100] **100** → **100**	[55,57] [76,88] 56 → 82	[0,99] [100,100] 50 → **100**	[100,100] [100,100] **100** → **100**	[82,94] [100,100] 88 → **100**	[100,100] [100,100] **100** → **100**
	task4	[0,0] [0,0] 0 → 0	[14,44] [100,100] 29 → **100**	[98,100] [100,100] **100** → **100**	[98,100] [100,100] 98 → **100**	[98,100] [100,100] **100** → **100**	[30,48] [56,76] 37 → 66	[25,100] [100,100] 75 → **100**	[100,100] [100,100] **100** → **100**	[94,97] [100,100] 95 → **100**	[100,100] [100,100] **100** → **100**
	task5	[0,0] [0,0] 0 → 0	[20,74] [100,100] 54 → **100**	[100,100] [100,100] **100** → **100**	[95,100] [100,100] 97 → **100**	[79,100] [100,100] 91 → **100**	[38,55] [98,100] 47 → **100**	[0,11] [100,100] 4 → **100**	[98,100] [100,100] **100** → **100**	[99,98] [100,100] 94 → **100**	[100,100] [100,100] **100** → **100**
	avg. (5 tasks)	[0,0] [19,20] 0 → 20	[47,65] [100,100] 55 → **100**	[99,100] [100,100] **100** → **100**	[96,99] [100,100] 98 → **100**	[95,100] [100,100] 98 → **100**	[55,61] [89,92] 58 → 90	[48,76] [100,100] 63 → **100**	[99,100] [100,100] **100** → **100**	[92,98] [100,100] 95 → **100**	[100,100] [100,100] **100** → **100**
scene-sparse	task1	[0,2] [96,99] 1 → 97	[0,9] [100,100] 4 → **100**	[55,79] [100,100] 69 → **100**	[97,100] [100,100] 99 → **100**	[19,25] [100,100] 22 → **100**	[52,74] [100,100] 64 → **100**	[98,100] [100,100] 99 → **100**	[100,100] [100,100] **100** → **100**	[98,100] [100,100] 98 → **100**	[100,100] [100,100] **100** → **100**
	task2	[0,0] [94,100] 0 → 97	[30,78] [100,100] 51 → **100**	[41,64] [98,100] 51 → 99	[97,100] [98,100] 99 → 99	[2,6] [33,51] 4 → 42	[7,23] [98,100] 15 → 98	[82,94] [100,100] 88 → **100**	[96,100] [100,100] 98 → **100**	[97,100] [100,100] 98 → **100**	[100,100] [98,100] **100** → **100**
	task3	[0,0] [0,0] 0 → 0	[0,2] [98,100] 0 → 99	[48,83] [100,100] 68 → **100**	[90,96] [100,100] 93 → **100**	[0,2] [9,40] 1 → 24	[21,47] [95,99] 34 → 97	[96,100] [100,100] 98 → **100**	[83,96] [100,100] 90 → **100**	[74,89] [91,100] 82 → 96	[98,100] [100,100] **100** → **100**
	task4	[0,0] [0,0] 0 → 0	[0,0] [98,100] 0 → 98	[69,81] [98,100] 75 → 96	[71,99] [99,99] 86 → 97	[41,63] [86,99] 50 → 94	[85,96] [85,100] 92 → 94	[85,98] [95,100] 92 → 97	[87,96] [98,100] 93 → 99	[98,100] [84,96] **100** → 89	[98,100] [100,100] **100** → **100**
	task5	[0,0] [0,0] 0 → 0	[0,0] [95,98] 0 → 97	[18,32] [73,88] 25 → 81	[14,52] [98,100] 38 → 99	[12,17] [78,98] 14 → 91	[58,82] [97,100] 71 → 98	[34,50] [98,100] 41 → 99	[14,70] [91,100] 43 → 95	[82,91] [80,97] 87 → 89	[96,98] [98,100] **97** → **100**
	avg. (5 tasks)	[0,0] [39,39] 0 → 39	[7,16] [99,99] 11 → 99	[55,60] [93,98] 57 → 95	[82,87] [99,100] 85 → 99	[17,21] [64,76] 18 → 70	[50,64] [95,99] 57 → 97	[81,87] [99,100] 84 → 99	[79,89] [98,100] 84 → 99	[91,95] [91,98] 93 → 95	[99,100] [100,100] **99** → **100**
cube-double	task1	[0,2] [0,0] 1 → 0	[8,16] [98,100] 12 → 99	[50,76] [97,100] 63 → 99	[55,86] [94,100] 70 → 97	[22,43] [97,100] 33 → 99	[22,24] [98,100] 23 → **100**	[52,77] [100,100] 66 → **100**	[79,90] [98,100] 84 → 99	[89,95] [96,100] 92 → 98	[97,100] [100,100] **99** → **100**
	task2	[0,0] [0,0] 0 → 0	[0,0] [4,28] 0 → 16	[24,45] [90,94] 36 → 92	[70,85] [85,92] 79 → 88	[4,10] [94,97] 8 → 96	[6,8] [56,75] 8 → 65	[31,61] [100,100] 43 → **100**	[73,84] [100,100] 79 → **100**	[60,82] [88,96] 71 → 92	[89,96] [98,100] **92** → **100**
	task3	[0,0] [0,0] 0 → 0	[0,0] [14,54] 0 → 34	[10,34] [94,99] 22 → 96	[82,85] [84,91] 83 → 87	[0,3] [88,96] 1 → 92	[4,10] [60,84] 7 → 73	[23,52] [98,100] 38 → **99**	[56,80] [98,100] 68 → **99**	[41,68] [94,98] 54 → 96	[87,96] [98,100] **91** → **99**
	task4	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[2,15] [0,2] 9 → 1	[14,29] [28,45] 22 → 38	[0,0] [0,4] 0 → 2	[0,4] [0,0] 1 → 0	[8,15] [100,100] 11 → **100**	[12,35] [85,99] 22 → 92	[3,14] [43,49] 8 → 46	[44,48] [98,100] **45** → **100**
	task5	[0,0] [0,0] 0 → 0	[0,2] [0,2] 1 → 1	[7,17] [90,94] 12 → 92	[78,87] [94,98] 82 → 96	[5,20] [93,99] 13 → 96	[12,15] [65,88] 13 → 78	[32,46] [98,100] 37 → **99**	[68,80] [97,100] 74 → **99**	[53,63] [86,96] 57 → 92	[89,94] [97,100] **91** → **99**
	avg. (5 tasks)	[0,0] [0,0] 0 → 0	[2,3] [28,32] 3 → 30	[23,34] [75,76] 29 → 76	[63,73] [78,81] 68 → 80	[7,13] [76,77] 11 → 77	[10,13] [62,69] 11 → 65	[32,47] [100,100] 39 → **100**	[64,71] [96,99] 67 → 98	[50,64] [83,86] 56 → 85	[82,87] [99,100] **84** → **100**
cube-triple	task1	[0,0] [0,2] 0 → 1	[0,2] [0,2] 1 → 1	[3,8] [78,90] 5 → 87	[11,23] [95,98] 17 → 97	[0,0] [0,6] 0 → 3	[0,4] [0,3] 2 → 1	[13,26] [100,100] 19 → **100**	[8,19] [100,100] 13 → **100**	[17,29] [100,100] 22 → **100**	[45,78] [100,100] **60** → **100**
	task2	[0,0] [0,0] 0 → 0	[0,2] [0,0] 1 → 0	[0,2] [0,26] 1 → 8	[0,2] [0,26] 1 → 8	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,2] [84,93] 1 → 89	[0,2] [82,94] 1 → 89	[0,0] [65,72] 0 → 68	[1,8] [99,100] **4** → **99**
	task3	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,2] [0,12] 0 → 5	[0,3] [5,18] 1 → 11	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [29,70] 0 → 51	[1,14] [70,78] 7 → 73	[0,2] [64,72] 1 → 68	[15,26] [98,100] **20** → **99**
	task4	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,2] 0 → 1	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [10,52] 0 → 26	[0,0] [40,65] 0 → 54	[0,0] [9,24] 0 → 16	[1,12] [97,100] **6** → **98**
	task5	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [88,98] 0 → **94**	[1,4] [80,98] **2** → 90
	avg. (5 tasks)	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[1,2] [16,20] 1 → 18	[3,5] [21,27] 4 → 23	[0,0] [0,1] 0 → 1	[0,1] [0,1] 0 → 0	[3,5] [46,61] 4 → 53	[3,10] [61,66] 6 → 64	[4,6] [66,73] 4 → 69	[14,25] [95,99] **19** → **97**
cube-quadruple	task1	[0,0] [0,0] 0 → 0	[2,10] [97,100] 7 → 98	[0,2] [2,21] 1 → 14	[5,10] [52,64] 7 → 56	[23,33] [95,99] 28 → **97**	[0,0] [0,0] 0 → 0	[3,10] [99,100] 6 → 98	[6,27] [98,100] 16 → 99	[51,63] [71,91] **57** → 81	[5,42] [98,100] 20 → **100**
	task2	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,8] 0 → 3	[0,3] [39,59] 2 → 49	[0,0] [0,0] 0 → 0	[0,0] [96,99] 0 → 97	[0,0] [98,100] 0 → 98	[12,26] [15,20] **20** → 18	[0,2] [98,100] 0 → **99**
	task3	[0,0] [0,0] 0 → 0	[0,0] [0,2] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,2] 0 → 0	[4,7] [2,6] **5** → 4	[0,0] [0,0] 0 → 0	[0,0] [91,96] 0 → **94**	[0,8] [74,82] 3 → 78	[2,4] [0,5] 2 → 2	[1,8] [62,76] 4 → 69
	task4	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,3] [18,46] **1** → 32	[0,0] [0,0] 0 → 0	[0,0] [91,97] 0 → 94	[0,2] [82,95] **1** → 88	[0,0] [0,3] 0 → 2	[0,0] [94,96] 0 → **96**
	task5	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[0,0] [0,0] 0 → 0	[9,19] [6,14] **13** → 9	[0,0] [0,0] 0 → 0
	avg. (5 tasks)	[0,0] [0,0] 0 → 0	[0,2] [20,20] 1 → 20	[0,0] [0,6] 0 → 3	[1,2] [12,13] 1 → 12	[6,8] [36,37] 7 → 36	[0,0] [0,0] 0 → 0	[1,2] [76,77] 1 → **77**	[2,7] [72,75] 4 → 74	[16,21] [20,25] **18** → 22	[1,10] [71,74] 5 → 73
overall	avg. (25 tasks)	[0,0] [12,12] 0 → 12	[13,15] [49,50] 14 → 50	[36,38] [57,60] 37 → 58	[50,52] [62,63] 51 → 63	[25,28] [56,58] 27 → 57	[24,26] [48,52] 25 → 50	[35,41] [84,88] 38 → 86	[50,53] [86,87] 52 → 86	[50,56] [72,77] 53 → 74	[59,64] [93,94] **61** → **94**

Table 6: **Complete OGBench offline-to-online RL results by task.** For each cell, we report the offline performance after 1M of training steps and then the online performance after 1M of additional online steps. The best method(s) for each column is highlighted in bold and color. All values are means over 4 seeds with 95% confidence intervals.

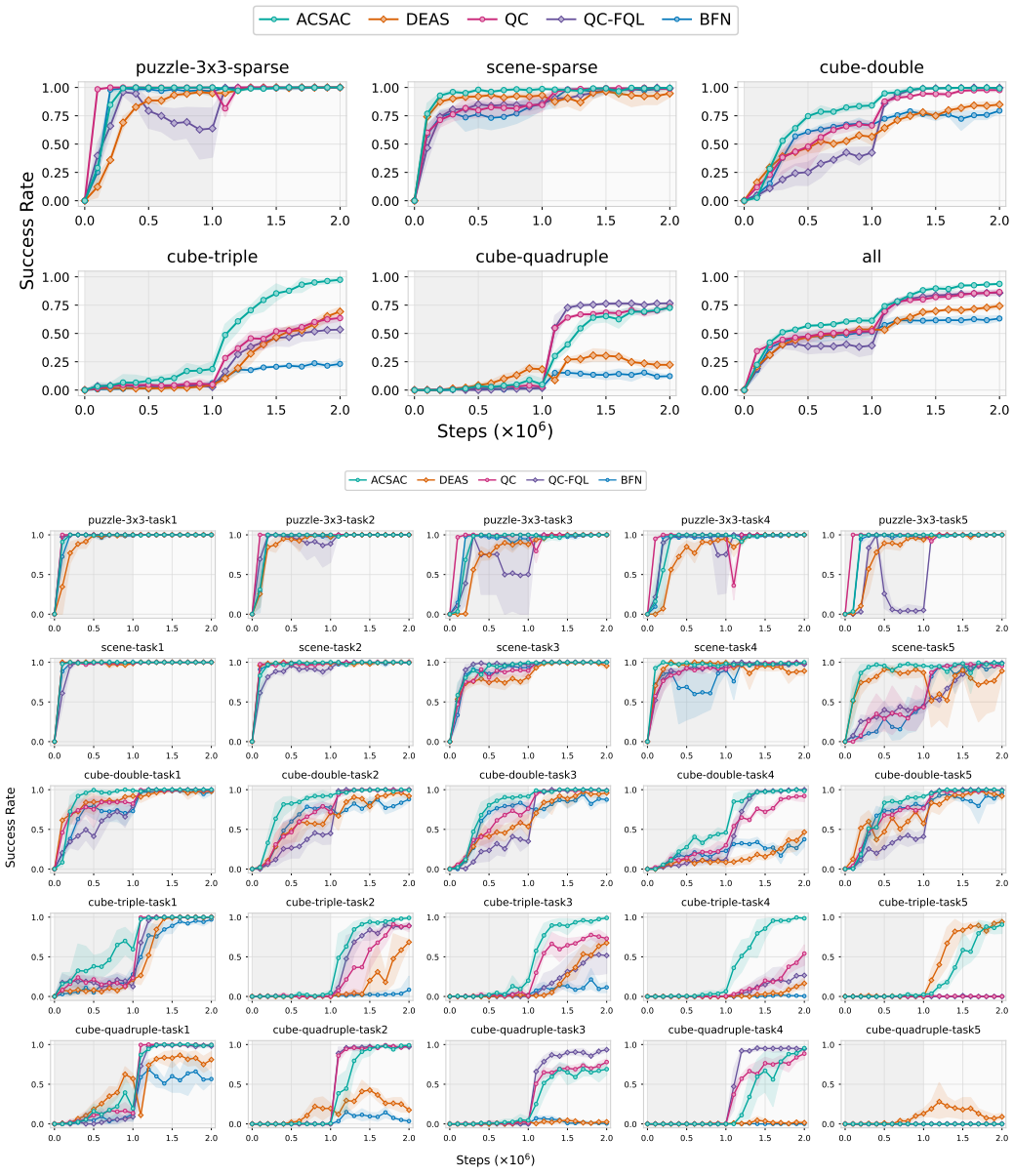

Figure 6: **Complete OGBench offline-to-online RL results by task.** Following Li et al. [21]'s appendix convention, the figure first shows summary plots by domain (corresponding to Table 1) and then per-task curves (corresponding to Table 6). The first 1M steps correspond to offline training and the next 1M steps correspond to online fine-tuning. Results are averaged over 4 seeds and plotted with 95% confidence intervals.

F.3 Offline RL Results for OGBench

Table 7 reports an offline-only comparison on OGBench manipulation, complementary to the offline-to-online results in Table 1. For tasks already evaluated in prior work, we use the reported numbers directly (entries without $\pm$). The `FQL`, `FQL-n`, `QC-FQL`, `DEAS`, `DQC`, and `CGQ` entries are taken from Song et al. [36]. The evaluation protocol is described in Appendix E.2.

Task Category	FQL	FQL-n	QC-FQL	DEAS	DQC	CGQ	ACSAC
scene-sparse (5 tasks)	57	18	84	93	81	86	$\mathbf{98 \pm 1}$
cube-double (5 tasks)	29	11	39	48	31	69	$\mathbf{81 \pm 4}$
puzzle-3x3-sparse (5 tasks)	**100**	**98**	63	**99**	**100**	**99**	$\mathbf{100 \pm 0}$
puzzle-4x4 (5 tasks)	17	23	26	**39**	8	28	33 ± 2
Average (Manipulation)	51	38	53	70	55	71	$\mathbf{78 \pm 2}$

Table 7: **Offline RL results on OGBench manipulation benchmarks.** Following Song et al. [36], we highlight results within 95% of the best performance in bold.

G Theoretical Foundations

We analyze ACSAC as behavior-constrained variable-duration Bellman learning. A length-H chunk drawn from the flow BC policy is treated as a proposal path. At each replanning state, ACSAC may execute any prefix of this path and replan afterwards. We establish four results. First, prefix truncation is deterministic post-processing, so it cannot increase the action-level mismatch between the flow BC policy and the behavior chunk distribution at the corresponding prefix length. Second, prefix-conditioned Q-values of different lengths are well-defined under prefix consistency, and they share the unit of total discounted return at the unrestricted variable-length Bellman optimum. Third, the critic loss in Equation 6 approximates a γ-contractive Bellman backup whose unique fixed point is the action-value function of the deployed policy $\pi_\star$. Fourth, averaging per-horizon squared losses at the gradient level reduces update variance under the multi-step targets, while preserving the sparse reward signal in each per-horizon target.

Throughout this appendix we work in the deterministic setting and at the level of exact expectations. A short final paragraph in Section G.5 sketches what changes in stochastic environments and under finite-sample approximation, and points to DQC and EMaQ for the heavier analysis.

G.1 Setup and Notation

Assumption G.1 (Deterministic MDP for analysis)**.** For the proofs we consider the deterministic version of the MDP in Section 3, with $s_{t+1} = f(s_t, a_t)$ and bounded rewards $|r(s,a)| \le R_{\max}$, $\gamma \in (0,1)$. We use the sup-norm $\|Q\|_\infty := \sup_{s,a_{t:t+h}} |Q(s, a_{t:t+h})|$ on bounded Q-functions, and write the proofs for finite or discretized state-action spaces. The same contraction arguments extend to bounded continuous spaces when the displayed maxima exist; otherwise, maxima can be replaced by suprema. The OGBench tasks evaluated in Section 5 are deterministic.

Variable-length action chunks. We use the body's chunk notation $a_{t:t+h} := (a_t, a_{t+1}, \dots, a_{t+h-1}) \in \mathcal{A}^h$ throughout this appendix, with $h \in [H] := \{1, \dots, H\}$ and $[N] := \{1, \dots, N\}$. We write

$$\mathcal{A}^{\le H} := \bigcup_{h=1}^{H} \mathcal{A}^h \tag{9}$$

for the set of executable prefixes, with the convention that an element carries its own length h. Under Assumption G.1, executing $a_{t:t+h}$ from s_t produces the unique trajectory $s_{t+1} = f(s_t, a_t), \dots, s_{t+h} = f(s_{t+h-1}, a_{t+h-1})$, with open-loop chunk return

$$r_t^{(h)} := \sum_{\tau=0}^{h-1} \gamma^\tau \, r(s_{t+\tau}, a_{t+\tau}), \tag{10}$$

in agreement with the prefix reward used in the chunked TD loss of Equation 2.

Behavior and proposal distributions. Let $\pi_\beta^H(\cdot \mid s_t) \in \Delta(\mathcal{A}^H)$ denote the true behavior chunk distribution under the data-collection policy of $\mathcal{D}$, and let $\pi_\theta^H(\cdot \mid s_t) \in \Delta(\mathcal{A}^H)$ denote the full-length law of the flow BC policy of Section 4.3. For $h \in [H]$, $\pi_\theta^h(\cdot \mid s_t)$ denotes the distribution of the first h actions when a full chunk is sampled from $\pi_\theta^H(\cdot \mid s_t)$,

$$\pi_\theta^h(a_{t:t+h} \mid s_t) \;:=\; \int_{\mathcal{A}^{H-h}} \pi_\theta^H(a_{t:t+H} \mid s_t)\, da_{t+h:t+H}, \tag{11}$$

and π_β^h is defined analogously from π_β^H. The integral becomes a sum in the discrete case.

Equal-value candidates. When several candidates attain the same maximum value, we choose one using a fixed state-independent rule, such as the first candidate in the stored order. The conclusions below do not depend on how ties are resolved; the rule only makes the policy extraction in Definition G.14 below single-valued.

G.2 Prefix Truncation Does Not Increase Behavior Mismatch

ACSAC's bootstrap target queries the critic at an adaptively-chosen prefix of a behavior-generated chunk. A natural concern is whether shorter prefixes drift further from the behavior support than the full chunk does. We rule this out at the level of total variation distance, using only the elementary fact that marginalization is non-expansive in TV.

Lemma G.2 (Marginalization is TV non-expansive)**.** *For any two distributions μ, ν on $\mathcal{A}^H$ and any $h \in [H]$, let μ_h, ν_h denote their length-h prefix marginals as in Equation 11. Then*

$$D_{\mathrm{TV}}(\mu_h, \nu_h) \;\le\; D_{\mathrm{TV}}(\mu, \nu). \tag{12}$$

Proof. Using the convention $D_{\mathrm{TV}}(p, q) = \frac{1}{2}\sum_x |p(x) - q(x)|$ and the triangle inequality, $\sum_{a_{t:t+h}} \left|\sum_{a_{t+h:t+H}} (\mu - \nu)(a_{t:t+H})\right| \le \sum_{a_{t:t+H}} |\mu(a_{t:t+H}) - \nu(a_{t:t+H})|$. Dividing by 2 gives Equation 12. The continuous case is analogous, with sums replaced by integrals. □

Theorem G.3 (Prefix truncation does not increase behavior mismatch)**.** *Let $\delta_\theta(s_t) := D_{\mathrm{TV}}(\pi_\theta^H(\cdot \mid s_t), \pi_\beta^H(\cdot \mid s_t))$ be the full-chunk behavior mismatch of the flow BC policy at state s_t. Then for every $h \in [H]$,*

$$D_{\mathrm{TV}}\big(\pi_\theta^h(\cdot \mid s_t),\ \pi_\beta^h(\cdot \mid s_t)\big) \;\le\; \delta_\theta(s_t). \tag{13}$$

Proof. Apply Lemma G.2 with $\mu = \pi_\theta^H(\cdot \mid s_t)$ and $\nu = \pi_\beta^H(\cdot \mid s_t)$. □

Remark G.4 (Implication for the TD target). Theorem G.3 concerns the proposal distribution before value-based selection. Shortening a full behavior-generated chunk to any prefix does not increase its action-level mismatch from the corresponding behavior prefix marginal. The set of prefixes used by the bootstrap target in Equation 6 is therefore obtained from behavior-generated chunks without any length-dependent shift caused by truncation itself.

Remark G.5 (Selection adds a separate, length-independent cost). Value-based rejection sampling does change the selected action distribution. This is the standard rejection-sampling cost already present in QC and EMaQ-style schemes: QC's closed-form bound [21] gives $D_{\mathrm{KL}}(\pi^H_{N\text{-best}} \,\|\, \pi_\theta^H) \le \log N - (N-1)/N$ for the rejection-sampled chunk, and the data-processing inequality propagates the same bound to any prefix length, so the selection cost is independent of which length is eventually executed. ACSAC therefore separates the selection cost (controlled by N) from the prefix-truncation TV bound (Theorem G.3); we do not chain the two bounds, since KL divergence does not satisfy a triangle inequality. This statement is action-level only and is not an OOD detector; state-distribution shift under open-loop execution is a separate issue, briefly discussed at the end of Section G.5.

G.3 Prefix Values Are Well-Defined and Cross-Horizon Comparable

For ACSAC's joint argmax over (n, h) to be meaningful, two conditions are needed. The network output at the h-th position must represent a well-defined Q-value of the length-h prefix, and the resulting Q-values across prefix lengths must lie on a single comparable scale.

Definition G.6 (Prefix consistency)**.** A network $\hat{Q} : \mathcal{S} \times \mathcal{A}^H \to \mathbb{R}^H$ is *prefix-consistent* if its h-th output takes the same value on any two length-H chunks that share the same first h actions. Equivalently, $\hat{Q}^{(h)}(s, a_{t:t+H})$ depends only on $(s, a_{t:t+h})$ and not on the suffix $a_{t+h:t+H}$.

Proposition G.7 (Causal-masked Transformer is sufficient)**.** *A decoder-only Transformer with causal self-attention and one output head per position satisfies prefix consistency.*

Proof. Index the input tokens as $(s, a_t, \ldots, a_{t+H-1})$ with action a_{t+h-1} at position h. The causal mask restricts the hidden representation at position h to attend only to positions $0, \ldots, h$, and induction over layers preserves this restriction. The h-th output head reads only this hidden representation, so its value depends only on $(s, a_{t:t+h})$. □

Remark G.8 (Sufficient, not necessary). A generic MLP that takes the entire chunk as input does not enforce prefix consistency by architecture. It could in principle learn the suffix-invariance from data, but the property is not guaranteed. ACSAC relies on the causal Transformer for this property by construction, not on data-driven invariance.

Variable-horizon Bellman semantics. For any continuation policy π and any state s_t, the variable-horizon prefix value is defined as

$$Q^\pi(s_t, a_{t:t+h}) \;:=\; r_t^{(h)} \;+\; \gamma^h \, V^\pi(s_{t+h}), \tag{14}$$

where V^π is the standard state-value function of π in the original one-step MDP. This is the discounted return of committing to the prefix $a_{t:t+h}$ open-loop and then following π from s_{t+h}. All prefix lengths therefore share the same quantity type, namely the total discounted return from s_t, with prefix consistency ensuring that each output of a prefix-consistent network estimates this quantity for a unique length-h prefix.

Theorem G.9 (Prefix Q-values are cross-horizon comparable)**.** *For any continuation policy π, Q^π in Equation 14 is the action-value function of the variable-horizon Bellman backup*

$$(\mathcal{B}^\pi Q)(s_t, a_{t:t+h}) \;:=\; r_t^{(h)} + \gamma^h \, V^\pi(s_{t+h}), \tag{15}$$

which is constant in Q and therefore trivially has Q^π as its unique fixed point. For a full chunk $a_{t:t+H}$ and any $h_1 \le h_2 \in [H]$, writing the additional reward over the segment $[h_1, h_2)$ as

$$r_{t+h_1}^{(h_2-h_1)} \;:=\; \sum_{j=0}^{h_2-h_1-1} \gamma^j r(s_{t+h_1+j}, a_{t+h_1+j}), \tag{16}$$

we obtain the difference identity

$$\begin{aligned} &Q^\pi(s_t, a_{t:t+h_2}) - Q^\pi(s_t, a_{t:t+h_1}) \\ &\quad= \; \gamma^{h_1}\Big(r_{t+h_1}^{(h_2-h_1)} + \gamma^{h_2-h_1} \, V^\pi(s_{t+h_2}) - V^\pi(s_{t+h_1})\Big). \end{aligned} \tag{17}$$

Proof. Equation 14 is the definition; substitute the two cases $h = h_1, h_2$ and factor γ^{h_1} to obtain Equation 17 after splitting $r_t^{(h_2)} - r_t^{(h_1)} = \gamma^{h_1} r_{t+h_1}^{(h_2-h_1)}$. □

Remark G.10 (Replanning-horizon interpretation). A longer prefix is preferred whenever the extra open-loop segment plus delayed continuation exceeds the value of replanning at s_{t+h_1}. ACSAC's joint argmax over (n, h) in Equation 8 therefore compares replanning horizons on a common return scale. Specializing π to the optimal one-step policy $\pi^\star$ yields the corollary

$$Q^{\pi^\star}(s_t, a_{t:t+h}) \;=\; r_t^{(h)} + \gamma^h \, V^\star(s_{t+h}), \tag{18}$$

where $V^\star$ is the optimal value function of the original one-step MDP. This identity also identifies $Q^{\pi^\star}$ with the fixed point of the unrestricted variable-horizon Bellman optimality operator $(\mathcal{B}^\star_{\le H} Q)(s_t, a_{t:t+h}) := r_t^{(h)} + \gamma^h \max_{a' \in \mathcal{A}^{\le H}} Q(s_{t+h}, a')$.

G.4 Expected-Prefix-Max Bellman Backup

We now show that the per-horizon target inside the critic loss in Equation 6 is an unbiased Monte Carlo sample of an expected-prefix-max Bellman backup $\mathcal{B}_\theta^{N,H}$ that is a γ-contraction in sup-norm. Its unique fixed point is the action-value function of the deployed adaptive-prefix policy $\pi_\star$. The structure mirrors EMaQ [10], which samples N behavior actions and backs up their max; ACSAC samples N behavior chunks and backs up the max over all NH executable prefixes.

Definition G.11 (ACSAC expected-prefix-max Bellman backup)**.** For bounded $Q : \mathcal{S} \times \mathcal{A}^{\leq H} \to \mathbb{R}$, the *ACSAC Bellman backup* is

$$\begin{aligned}(\mathcal{B}_\theta^{N,H} Q)(s_t, a_{t:t+h}) \;&:=\; r_t^{(h)} \\ &+\; \gamma^h \, \mathbb{E}_{\tilde{a}_{t+h:t+h+H}^{(1:N)} \sim \pi_\theta^H(\cdot|s_{t+h})} \left[\max_{n\in[N],\, k\in[H]} Q\Big(s_{t+h},\, \tilde{a}_{t+h:t+h+k}^{(n)}\Big) \right].\end{aligned} \tag{19}$$

The tilde distinguishes the N i.i.d. proposal chunks at the bootstrap state s_{t+h} from the chunk $a_{t:t+H}$ on the regression side, and the maximum ranges over the NH candidate prefixes formed from these proposals.

Lemma G.12 (Max non-expansiveness over a common index set)**.** *For any finite index set $\mathcal{I}$ and any real-valued $g_1, g_2 : \mathcal{I} \to \mathbb{R}$,*

$$\big|\max_{i\in\mathcal{I}} g_1(i) - \max_{i\in\mathcal{I}} g_2(i)\big| \;\leq\; \max_{i\in\mathcal{I}} |g_1(i) - g_2(i)|. \tag{20}$$

Proof. Let $i_1 \in \arg\max_i g_1(i)$ and $i_2 \in \arg\max_i g_2(i)$. Then $\max_i g_1(i) - \max_i g_2(i) \leq g_1(i_1) - g_2(i_1) \leq |g_1(i_1) - g_2(i_1)| \leq \max_i |g_1(i) - g_2(i)|$, and the symmetric direction yields the absolute value. This is a deterministic, set-theoretic inequality, so it holds even when the indexed values $g_1(i), g_2(i)$ are correlated across i, which is what allows EMaQ's contraction argument to transport to ACSAC's NH correlated prefix candidates [10]. □

Theorem G.13 (Contraction and unique fixed point)**.** *The operator $\mathcal{B}_\theta^{N,H}$ in Equation 19 satisfies*

$$\|\mathcal{B}_\theta^{N,H} Q_1 - \mathcal{B}_\theta^{N,H} Q_2\|_\infty \;\leq\; \gamma \, \|Q_1 - Q_2\|_\infty \tag{21}$$

for all bounded $Q_1, Q_2 : \mathcal{S} \times \mathcal{A}^{\leq H} \to \mathbb{R}$, so $\mathcal{B}_\theta^{N,H}$ has a unique fixed point in the space of bounded Q-functions.

Proof. Fix $(s_t, a_{t:t+h})$ and let $s' = s_{t+h}$. The shared $r_t^{(h)}$ cancels, so $|(\mathcal{B}_\theta^{N,H} Q_1)(s_t, a_{t:t+h}) - (\mathcal{B}_\theta^{N,H} Q_2)(s_t, a_{t:t+h})| = \gamma^h \big|\mathbb{E}[\max_{n,k} Q_1(s', \cdot)] - \mathbb{E}[\max_{n,k} Q_2(s', \cdot)]\big|$. By Jensen's inequality and Lemma G.12 applied pointwise to each realized proposal sample, $\big|\mathbb{E}[\max Q_1] - \mathbb{E}[\max Q_2]\big| \leq \mathbb{E}\big[\max_{n,k} |Q_1(s', \cdot) - Q_2(s', \cdot)|\big] \leq \|Q_1 - Q_2\|_\infty$. Hence $|(\mathcal{B}_\theta^{N,H} Q_1) - (\mathcal{B}_\theta^{N,H} Q_2)| \leq \gamma^h \|Q_1 - Q_2\|_\infty \leq \gamma \, \|Q_1 - Q_2\|_\infty$. A sup-norm contraction has a unique fixed point, which we denote $Q_\theta^{N,H}$. □

Definition G.14 (Critic-induced extraction policy)**.** For any bounded Q, the extraction policy $\pi_\star^Q$ acts at state s_t as follows. Draw $a_{t:t+H}^{(1)}, \ldots, a_{t:t+H}^{(N)} \overset{\text{i.i.d.}}{\sim} \pi_\theta^H(\cdot \mid s_t)$, set

$$(n^\star, h^\star) \;=\; \arg\max_{n\in[N],\, h\in[H]} Q\Big(s_t,\, a_{t:t+h}^{(n)}\Big), \tag{22}$$

and execute the prefix $a_{t:t+h^\star}^{(n^\star)}$, treating it as one temporally extended action. We write $\pi_\star := \pi_\star^{Q_\theta^{N,H}}$ for the idealized adaptive-prefix policy induced by the operator fixed point; the deployed implementation in Section 4.3 applies the same extraction rule to the learned critic Q_ϕ.

Theorem G.15 (Fixed-point identity)**.** *$Q_\theta^{N,H}$ is the action-value function of $\pi_\star$ when each selected prefix is treated as one temporally extended action; in particular $Q_\theta^{N,H} = Q^{\pi_\star}$.*

Proof. Equation 22 gives, for any draw of the N proposals, $Q_\theta^{N,H}(s_t, a_{t:t+h^\star}^{(n^\star)}) = \max_{n\in[N],\,k\in[H]} Q_\theta^{N,H}(s_t, a_{t:t+k}^{(n)})$. Taking the expectation over the proposal draws, the inner expected max in Equation 19 equals $\mathbb{E}_{\pi_\star}[Q_\theta^{N,H}(s_t,\cdot)]$, so the fixed-point equation $Q_\theta^{N,H} = \mathcal{B}_\theta^{N,H} Q_\theta^{N,H}$ reads

$$Q_\theta^{N,H}(s_t, a_{t:t+h}) \;=\; r_t^{(h)} + \gamma^h\, \mathbb{E}_{a'\sim\pi_\star(\cdot|s_{t+h})}\big[Q_\theta^{N,H}(s_{t+h}, a')\big].$$

This is the variable-horizon Bellman equation for $\pi_\star$ and uniquely identifies its action-value function by the same contraction argument as Theorem G.13, with the deterministic selection $\pi_\star$ in place of the joint max. We may therefore write the fixed point as $Q^{\pi_\star}$ in the rest of the appendix. □

Proposition G.16 (Critic targets are Monte Carlo Bellman samples)**.** *Fix any bounded $Q : \mathcal{S} \times \mathcal{A}^{\le H} \to \mathbb{R}$ and define the per-horizon target*

$$\hat{G}_h(Q) \;:=\; r_t^{(h)} \;+\; \gamma^h \max_{n\in[N],\,k\in[H]} Q\Big(s_{t+h},\, \tilde{a}_{t+h:t+h+k}^{(n)}\Big), \tag{23}$$

with $\tilde{a}_{t+h:t+h+H}^{(1)}, \dots, \tilde{a}_{t+h:t+h+H}^{(N)} \overset{\text{i.i.d.}}{\sim} \pi_\theta^H(\cdot \mid s_{t+h})$. $\hat{G}_h(Q)$ is the explicit-max sample form of the body's bootstrap target G_h in Equation 5, with $\pi_\star$ replaced by the joint $\arg\max$ and the bootstrap critic treated as a free argument; the body's $G_h(s_t, a_{t:t+h})$ corresponds to $\hat{G}_h(Q_{\bar{\phi}})$ at this conditioning point. Under Assumption G.1,

$$\mathbb{E}\Big[\hat{G}_h(Q)\,\Big|\, s_t, a_{t:t+h}\Big] \;=\; (\mathcal{B}_\theta^{N,H} Q)(s_t, a_{t:t+h}), \tag{24}$$

and the conditional squared-error loss decomposes as

$$\begin{aligned}&\mathbb{E}\Big[\big(Q_\phi(s_t, a_{t:t+h}) - \hat{G}_h(Q)\big)^2\,\Big|\, s_t, a_{t:t+h}\Big] \\ &\quad=\; \big(Q_\phi(s_t, a_{t:t+h}) - (\mathcal{B}_\theta^{N,H} Q)(s_t, a_{t:t+h})\big)^2 + \mathrm{Var}\Big[\hat{G}_h(Q)\,\Big|\, s_t, a_{t:t+h}\Big].\end{aligned} \tag{25}$$

Hence the empirical critic loss in Equation 6 is a Monte Carlo regression toward the Bellman backup $\mathcal{B}_\theta^{N,H} Q$, not an exact squared Bellman residual for any single sampled target. In the tabular exact-expectation regime, fitted iteration with $\mathcal{B}_\theta^{N,H}$ converges to $Q_\theta^{N,H}$.

Proof. Equation 24 follows from Definition G.11, since under Assumption G.1 the rewards $\{r_{t+\tau}\}_{\tau<h}$ summed in $r_t^{(h)}$ are deterministic given $(s_t, a_{t:t+h})$, and the only randomness in $\hat{G}_h(Q)$ comes from the proposal sampling that defines $\mathcal{B}_\theta^{N,H} Q$. The decomposition in Equation 25 is the standard bias-variance identity for the squared error of a deterministic prediction $Q_\phi(s_t, a_{t:t+h})$ against a stochastic target $\hat{G}_h(Q)$ with mean $(\mathcal{B}_\theta^{N,H} Q)(s_t, a_{t:t+h})$. □

Remark G.17 (Bounded fixed-point values)*.* With rewards in $[-R_{\max}, R_{\max}]$, the contraction in Theorem G.13 maps the interval $[-R_{\max}/(1-\gamma),\, R_{\max}/(1-\gamma)]$ to itself pointwise, so $Q_\theta^{N,H}$ is bounded by the reward scale uniformly in $(s_t, a_{t:t+h})$. This is an operator-level statement; the iterates of a learned neural critic may transiently exceed this interval without explicit clipping.

G.5 Per-Horizon Loss Averaging Stabilizes Updates

The multi-step targets G_h in Equation 6 accumulate rewards over h steps, and the variance of this cumulative sum tends to grow with h. ACSAC averages all H per-horizon squared losses at the gradient level, in line with T-SAC's gradient-level averaging [39] and TOP-ERL's multi-horizon Transformer supervision [20]. A standard variance-of-mean argument explains this design choice.

Cumulative-reward variance. For the cumulative-reward part of G_h, expansion gives

$$\mathrm{Var}\Big[r_t^{(h)}\Big] \;=\; \sum_{i=0}^{h-1}\sum_{j=0}^{h-1} \gamma^{i+j}\,\mathrm{Cov}(r_{t+i},\, r_{t+j}), \tag{26}$$

which typically increases with h, especially when γ is close to 1 and per-step rewards are positively correlated.

Per-horizon gradient. Let $\delta_h(\phi) := Q_\phi(s_t, a_{t:t+h}) - G_h$ denote the per-horizon Bellman residual, and let $g_h := \delta_h(\phi)\, \nabla_\phi Q_\phi(s_t, a_{t:t+h})$ denote its contribution to $\nabla_\phi \mathcal{L}(\phi)$. The averaged loss $\frac{1}{H}\sum_h \delta_h^2$ in Equation 6 produces the gradient $\bar{g} := \frac{1}{H}\sum_{h=1}^{H} g_h$, and let $\tilde{g}_h := g_h - \mathbb{E}[g_h]$ denote its centered version.

Lemma G.18 (Variance reduction by averaging)**.** *Suppose the centered per-horizon gradients satisfy* $\mathbb{E}[\|\tilde{g}_h\|^2] \le \sigma^2$ *for every* h *and* $\mathbb{E}[\langle \tilde{g}_h, \tilde{g}_{h'} \rangle] \le \rho\,\sigma^2$ *for some* $\rho \in [-1/(H-1), 1]$ *and every* $h \neq h'$*. Then the centered average* $\bar{\tilde{g}} := \frac{1}{H}\sum_h \tilde{g}_h$ *satisfies*

$$\mathrm{Var}(\bar{g}) \;:=\; \mathbb{E}\Big[\big\|\bar{\tilde{g}}\big\|^2\Big] \;\le\; \sigma^2\left[\rho + \frac{1-\rho}{H}\right]. \tag{27}$$

Whenever $\rho < 1$*, this is strictly less than* σ^2*, the variance bound for any single* g_h*.*

Proof. Expand $\|\bar{\tilde{g}}\|^2 = H^{-2}\sum_{h,h'}\langle \tilde{g}_h, \tilde{g}_{h'}\rangle$ and take expectations. The diagonal terms contribute $H^{-2}\cdot H\sigma^2 = \sigma^2/H$, and the off-diagonal terms contribute at most $H^{-2}\cdot H(H-1)\rho\sigma^2$, which sum to the right-hand side of Equation 27. □

Remark G.19 (Imperfect correlation in ACSAC). This is the setting targeted by T-SAC's gradient-level averaging argument [39]. The per-horizon gradients g_h in ACSAC share the same critic parameters ϕ, so adjacent horizons are positively correlated; they are not identical because the residuals δ_h use different cumulative-reward sums (longer horizons include the rewards of shorter ones plus extra discounted terms) and different bootstrap states s_{t+h}. Whenever $\rho < 1$ in this regime, Lemma G.18 gives strict variance reduction relative to single-horizon TD training, while target-level averaging would collapse the per-horizon signals.

Remark G.20 (Sparse reward signals are preserved). ACSAC averages per-horizon *losses*, not the targets themselves. A non-zero reward $r_{t+\tau}$ enters every G_h with $h > \tau$ as a discounted summand, so the sparse signal propagates into all $H - \tau$ residuals simultaneously. This contrasts with target-level averaging, which would replace the per-horizon targets by a single averaged scalar and could dilute sparse rewards [39].

Beyond the deterministic and exact-expectation case. The four results above are statements about action-level mismatch, prefix Q-value semantics, and an operator-level Bellman backup in a deterministic MDP. Three further directions are listed here as informal pointers. First, $H = 1$ recovers EMaQ over single actions [10], and the suboptimality of $Q_\theta^{N,H}$ relative to the unrestricted variable-horizon optimum is governed by the standard EMaQ-style proposal coverage analysis. Second, with finite critic and proposal approximation, the prefix-selection regret degrades by margin terms that scale linearly with the critic and proposal errors. Third, in stochastic MDPs the open-loop chunk return is no longer deterministic, and the resulting nominal-versus-actual gap is the open-loop consistency issue studied by DQC [22]; we therefore state the main proofs in the deterministic setting and treat the stochastic extension as a separate issue. None of these is required for the four results stated above.